

Quadrature Rule and Its Applications

Submitted to the
Department of Applied Mathematics
4th Year Honors Project

Course Title : Forth Year Honors Project
Course Code : AMTH-460

Submitted By

Name	:	XYZ
Class Roll Number	:	***
Registration Number	:	****
Admission Session	:	2020-2021
Academic Session	:	2023-2024

Project Supervisor

Dr. Md. Shafiqul Islam
Professor

Department of Applied Mathematics
Faculty of Science
University of Dhaka
Dhaka-1000, Bangladesh

Date: October 30, 2025

Abstract

The present study investigates six prominent numerical quadrature rules—Gauss–Legendre, Gauss–Lobatto, Gauss–Radau, Gauss–Laguerre, Gauss–Chebyshev, and Gauss–Hermite—for the efficient and accurate approximation of definite integrals. Each method is systematically derived from its corresponding orthogonal polynomial and weight function, ensuring exactness for polynomials of specific degrees. The study bridges theoretical foundations with practical computational applications, highlighting the efficiency, convergence behavior, and numerical stability of each quadrature rule under various node counts and interval types.

Through detailed numerical experiments, Gauss–Legendre and Gauss–Lobatto were found to be the most versatile and stable for general integration tasks, while Gauss–Radau shows high accuracy for moderate node counts but is sensitive to numerical instability at higher orders. Gauss–Laguerre, initially less accurate, improves with increasing nodes, albeit with a slower convergence rate, reflecting its sensitivity to semi-infinite domains. Gauss–Hermite slightly outperforms Gauss–Chebyshev, though both are primarily suitable for integrals with specific weight functions.

This work provides a systematic framework for selecting appropriate quadrature methods in applied mathematics, engineering, and computational sciences. The findings also highlight the importance of node selection, stability considerations, and the connection between quadrature rules and orthogonal polynomials, offering a foundation for future research in adaptive, high-order, and multi-dimensional integration techniques.

Contents

Introduction . . . 7
Literature Review . . . 8
Objectives of the Project . . . 10

1 Gauss–Legendre Quadrature Derivation **12**
1.1 Exactness of Gaussian Quadrature . . . 12
1.2 2-Point Rule . . . 14
1.2.1 System of Governing Equations . . . 14
1.2.2 Final 2-Point Formula . . . 14
1.3 3-Point Rule . . . 15
1.3.1 System of Equations . . . 15
1.3.2 Solving with Symmetry . . . 15
1.3.3 Final 3-Point Formula . . . 16
1.4 Error Analysis . . . 16
1.5 Code for Gaussian Quadrature . . . 17

2 Radau Quadrature **19**
2.1 3-Point Rule ($n = 3$) . . . 19
2.1.1 System of Governing Equations . . . 19
2.1.2 Solution . . . 20
2.1.3 Final 3-Point Formula . . . 20
2.2 Summary of Nodes and Weights . . . 20
2.3 Error Analysis . . . 20
2.3.1 Actionable Insight . . . 21
2.4 Code for Radau Quadrature . . . 21

3 Lobatto Quadrature Derivation **23**
3.1 3-Point Rule ($n = 3$) . . . 23
3.1.1 System of Governing Equations . . . 23
3.1.2 Solution . . . 24
3.1.3 Final 3-Point Formula (Simpson's Rule) . . . 24
3.2 Summary of Nodes and Weights . . . 24
3.3 Error Analysis . . . 24
3.3.1 Mean-Value Form . . . 25
3.3.2 Actionable Insight . . . 25

3.4 Code for Lobatto Quadrature . . . 25

4 Gauss–Hermite Quadrature Derivation 27
4.1 Transformation Method . . . 27
4.2 2-Point Rule ($n = 2$) . . . 28
4.2.1 System of Governing Equations . . . 28
4.2.2 Final 2-Point Formula . . . 28
4.3 3-Point Rule ($n = 3$) . . . 29
4.3.1 System of Governing Equations . . . 29
4.3.2 Solving with Symmetry . . . 29
4.3.3 Final 3-Point Formula . . . 30
4.4 Summary of Nodes and Weights . . . 30
4.5 Error Analysis . . . 31
4.6 Code for Gauss-Hermite Quadrature . . . 31

5 Gauss–Laguerre Quadrature Derivation 32
5.1 Transformation Method . . . 32
5.2 1-Point Rule ($n = 1$) . . . 33
5.2.1 System of Governing Equations . . . 33
5.2.2 Solution . . . 33
5.2.3 Final 1-Point Formula . . . 33
5.3 2-Point Rule ($n = 2$) . . . 33
5.3.1 System of Governing Equations . . . 34
5.3.2 Solution . . . 34
5.3.3 Final 2-Point Formula . . . 34
5.4 3-Point Rule ($n = 3$) . . . 35
5.5 Summary of Nodes and Weights . . . 35
5.6 Code for Gaussian Laguerre . . . 36
5.7 Error Analysis . . . 36
5.7.1 Actionable Insight . . . 36

6 Gauss–Chebyshev Quadrature Derivation 37
6.1 1-Point Rule ($n = 1$) . . . 37
6.1.1 System of Governing Equations . . . 37
6.1.2 Solution . . . 37
6.1.3 Final 1-Point Formula . . . 38
6.2 2-Point Rule ($n = 2$) . . . 38
6.2.1 System of Governing Equations . . . 38
6.2.2 Solution via Symmetry . . . 38
6.2.3 Final 2-Point Formula . . . 39
6.3 3-Point Rule ($n = 3$) . . . 39
6.3.1 Final 3-Point Formula (Corrected) . . . 39
6.4 Summary of Nodes and Weights . . . 39
6.5 Code for Gaussian Quadrature . . . 39

6.6 Error Analysis . . . 40
6.6.1 Actionable Insight . . . 41
6.7 Generalized Gaussian Quadrature Rules . . . 41

7 Applications of Gaussian Quadrature 42
7.1 Introduction . . . 42
7.2 General Lagrange Interpolation (arbitrary n) . . . 42
7.3 1D Linear Shape Functions ($n = 2$, denote by L_i) . . . 43
7.4 1D Quadratic Shape Functions ($n = 3$, denote by Q_i) . . . 44
7.5 2D Bilinear Shape Functions ($n = 4$, denote by N_i) . . . 45
7.6 Summary of notation and properties . . . 46
7.7 Evaluation of 1D integral . . . 47
7.8 Evaluation of line integral . . . 49
7.9 Evaluation of the Double Integral by Gaussian Quadrature . . . 53
7.9.1 Example-1(Rectangular Region) . . . 53
7.9.2 Example-2(Quadrilateral Region) . . . 57
7.9.3 Example-3(Triangular Region) . . . 62
7.9.4 Example-4(Pentagonal Region) . . . 65
7.9.5 Problems List: . . . 70
Conclusion . . . 72
Bibliograpy . . . 73

Introduction

Numerical integration, also known as quadrature, is a cornerstone of applied mathematics and computational science, providing efficient methods to approximate definite integrals when analytical solutions are unavailable or difficult to obtain [3]. The development of quadrature rules dates back to the early 20th century and has since become a critical tool in engineering, physics, and applied sciences for solving problems involving differential equations, finite element analysis, and mathematical modeling [1]. Quadrature methods reduce the evaluation of an integral to a weighted sum of function values at selected points, called nodes, allowing accurate approximations with minimal computational effort.

Gaussian-type quadrature rules are among the most widely used due to their high accuracy and rapid convergence. These methods achieve exact integration for polynomials up to a certain degree by carefully selecting nodes and weights based on the roots of orthogonal polynomials corresponding to a given weight function [2]. In other words, the nodes of Gaussian quadrature are the zeros of specific orthogonal polynomials, and the weights are determined such that the quadrature is exact for polynomials of degree up to $2n - 1$, where n is the number of nodes. This deep connection to orthogonal polynomials is what gives Gaussian quadrature its remarkable efficiency and accuracy.

This project focuses on six major quadrature methods: Gauss–Legendre, Gauss–Lobatto, Gauss–Radau, Gauss–Laguerre, Gauss–Chebyshev, and Gauss–Hermite [4, 5]. Each method is derived from its corresponding orthogonal polynomial and weight function, providing a robust mathematical framework for numerical integration. The Gauss–Legendre rule is particularly effective for finite intervals, while Gauss–Lobatto and Gauss–Radau incorporate boundary points to improve accuracy for certain problems. Gauss–Laguerre and Gauss–Hermite are specifically designed for semi-infinite and infinite domains, respectively, and Gauss–Chebyshev efficiently handles integrals with specific weight functions.

The study involves both theoretical derivation and computational implementation, allowing a detailed comparison of their accuracy, convergence behavior, and numerical stability under various conditions [6]. Through numerical experiments, the performance of each quadrature rule is evaluated across different numbers of nodes, highlighting the strengths and limitations of each approach. Particular attention is given to numerical instability, round-off errors, and convergence rates for high-order quadrature formulas.

The motivation behind this project is to provide a systematic understanding of Gaussian-type quadrature rules, to guide practitioners in selecting the most suitable method for different types of integrals, and to contribute practical insights for applications in applied mathematics, physics, and engineering. By bridging theoretical derivations with computational experiments, this work aims to enhance the reliability, efficiency, and applicability of numerical integration techniques in complex real-world problems.

Literature Review

Numerical integration, or quadrature, is a fundamental technique in applied mathematics, computational science, and engineering, allowing accurate approximation of definite integrals that are otherwise difficult or impossible to evaluate analytically. Among these, Gaussian quadrature rules are particularly efficient, employing the roots of orthogonal polynomials as nodes to achieve exactness for polynomials of degree up to $2n-1$ with only n evaluations [1].

The Gauss-Legendre quadrature is widely used for standard finite intervals due to its high accuracy and rapid convergence. Variants such as Gauss-Lobatto and Gauss-Radau incorporate one or both endpoints of the interval, improving precision for problems where boundary evaluation is crucial, especially in finite element analysis and spectral methods [2, 7].

For semi-infinite or infinite domains, Gauss-Laguerre and Gauss-Hermite quadratures are tailored to handle exponential and Gaussian weight functions, respectively. Gauss-Laguerre finds applications in quantum mechanics and probability theory, whereas Gauss-Hermite is widely used in statistical physics and stochastic computations [3]. Gauss-Chebyshev quadrature, optimized for $(1-x^2)^{-1/2}$ weights, is often employed in approximation theory and orthogonal expansions. Comparatively, Gauss-Hermite slightly outperforms Gauss-Chebyshev in most practical computations.

Recent studies have extended Gaussian quadrature to irregular and multidimensional domains. Hussain and Islam [4] proposed a generalized composite quadrature rule over polygonal regions, demonstrating the flexibility of Gaussian quadrature in handling non-standard geometries and composite domains. Islam and Hossain [5] investigated integration over arbitrary quadrilateral regions, while Islam and Hossain [6] developed methods for numerical integration over arbitrary triangular regions, providing further insight into extending Gaussian quadrature to complex 2D geometries. Additionally, Davis and Rabinowitz [8] discussed practical numerical quadrature techniques for challenging integrands and high-dimensional problems. Further theoretical insights and error analyses of Gaussian quadrature rules can be found in Gautschi [11] and Piessens et al. [12], who explore computational stability and algorithmic implementations for high-order integration.

While Gaussian quadrature is theoretically robust, high-order implementations may face numerical instability and round-off errors due to the sensitivity of high-degree polynomial roots. Adaptive and composite schemes divide the domain into subintervals with locally optimized nodes to improve accuracy and stability [9, 10]. Modern computational frameworks also emphasize the use of recurrence relations and high-precision arithmetic to enhance the stability of high-degree Gauss-Radau, Gauss-Lobatto, and Gauss-Laguerre rules.

Overall, the literature demonstrates that Gaussian quadrature rules are both mathematically rigorous and computationally versatile. Their performance depends on node selection, weight functions, and domain characteristics, making them essential for

numerical analysis, finite element computations, and large-scale engineering applications. Ongoing research continues to expand their use in multidimensional integration and complex computational problems, with adaptive, composite, and high-precision algorithms enhancing accuracy and stability. Building on these advances, our study in Quadrature Rule and Its Applications provides a comprehensive evaluation of six quadrature rules, offering practical insights to guide efficient and reliable numerical computations in applied mathematics, physics, and engineering.

Objectives of the Project

---

1. To conduct a comprehensive study of various numerical quadrature rules and their applications in approximating definite integrals that cannot be solved analytically.

2. To derive and analyze six Gaussian-based quadrature formulas—Gauss–Legendre, Gauss–Lobatto, Gauss–Radau, Gauss–Laguerre, Gauss–Chebyshev, and Gauss–Hermite—using the principles of orthogonal polynomials and their corresponding weight functions.

3. To understand the theoretical foundation of these quadrature rules and how orthogonality ensures exactness for specific polynomial degrees.

4. To investigate the accuracy, convergence behavior, and numerical stability of each quadrature method for a range of test functions and node counts.

5. To perform a comparative analysis among the six methods, identifying their strengths, limitations, and appropriate domains of application.

6. To examine the effect of increasing the number of nodes (n) on accuracy and error behavior, emphasizing the trade-off between precision and numerical stability.

7. To determine the most suitable quadrature rule for different types of integrands and weight functions commonly encountered in applied mathematics, physics, and engineering.

8. To provide practical insights and numerical evidence that contribute to the development of efficient and reliable computational techniques for solving complex integral problems.

Chapter 1

Gauss–Legendre Quadrature Derivation

1.1 Exactness of Gaussian Quadrature

For $N = 2n - 1$, there exist Gaussian points x_i and weights w_i such that

$$\int_{-1}^{1} x^k \, dx = \sum_{i=1}^{n} w_i \, (x_i)^k$$

where $k = 0, 1, 2, \ldots, N$.

Proof

Let the Gaussian points x_i be the roots of the Legendre polynomial of degree n, denoted $P_n(x)$. Let $P(x)$ be an arbitrary polynomial with degree $\leq 2n - 1$.

By the Division Algorithm for Polynomials, we can write $P(x)$ as:

$$P(x) = P_n(x)Q(x) + R(x)$$

where $Q(x)$ and $R(x)$ are polynomials with degrees $\leq n - 1$.

The definition of $P_n(x)$ based on its roots is:

$$P_n(x) = \prod_{i=1}^{n}(x - x_i)$$

Since the Gaussian points x_i are the roots, we have $P_n(x_i) = 0$. Evaluating $P(x)$ at these points yields:

$$P(x_i) = P_n(x_i)Q(x_i) + R(x_i) \implies P(x_i) = R(x_i)$$

This establishes the key relationship between the polynomial and the remainder.

Now, we integrate the polynomial $P(x)$ over the interval $[-1, 1]$:

$$\begin{aligned}\int_{-1}^{1} P(x)\,dx &= \int_{-1}^{1} [P_n(x)Q(x) + R(x)]\,dx \\ &= \int_{-1}^{1} P_n(x)Q(x)\,dx + \int_{-1}^{1} R(x)\,dx\end{aligned}$$

Since $P_n(x)$ is a Legendre polynomial (orthogonal with respect to the weight function $w(x) = 1$), and $\deg(Q) \leq n - 1$, the **Orthogonality property** ensures that the first integral is zero:

$$\int_{-1}^{1} P_n(x)Q(x)\,dx = 0$$

Therefore, the integral simplifies to:

$$\int_{-1}^{1} P(x)\,dx = \int_{-1}^{1} R(x)\,dx$$

The polynomial $R(x)$, with $\deg(R) \leq n - 1$, can be exactly integrated using the n-point quadrature rule because the weights w_i are chosen to exactly integrate all polynomials of degree $\leq n - 1$. Specifically, $R(x)$ can be expressed using the Lagrange basis polynomials $L_i(x)$:

$$\begin{aligned}\int_{-1}^{1} R(x)\,dx &= \int_{-1}^{1} \sum_{i=1}^{n} R(x_i)L_i(x)\,dx \\ &= \sum_{i=1}^{n} R(x_i) \int_{-1}^{1} L_i(x)\,dx \\ &= \sum_{i=1}^{n} R(x_i)w_i\end{aligned}$$

Finally, substituting $R(x_i) = P(x_i)$ from above, we get:

$$\int_{-1}^{1} P(x)\,dx = \sum_{i=1}^{n} P(x_i)w_i$$

This proves that the n-point Gaussian Quadrature rule is exact for an arbitrary polynomial $P(x)$ with degree $\leq 2n - 1$.

1.2 2-Point Rule

For the 2-point rule over the canonical interval $[-1, 1]$, the approximation is:

$$\int_{-1}^{1} f(x)\,dx \approx w_1 f(x_1) + w_2 f(x_2) \tag{1.1}$$

To find the four unknown parameters (w_1, w_2, x_1, x_2), we enforce that the rule is exact for polynomials of degree up to $2n - 1 = 3$. This gives us a system of four equations using the basis functions $1, x, x^2, x^3$.

1.2.1 System of Governing Equations

$$f(x) = 1: \quad \int_{-1}^{1} 1\,dx = 2 \quad \Longrightarrow \quad w_1 + w_2 = 2 \tag{1.2}$$

$$f(x) = x: \quad \int_{-1}^{1} x\,dx = 0 \quad \Longrightarrow \quad w_1 x_1 + w_2 x_2 = 0 \tag{1.3}$$

$$f(x) = x^2: \quad \int_{-1}^{1} x^2\,dx = \frac{2}{3} \quad \Longrightarrow \quad w_1 x_1^2 + w_2 x_2^2 = \frac{2}{3} \tag{1.4}$$

$$f(x) = x^3: \quad \int_{-1}^{1} x^3\,dx = 0 \quad \Longrightarrow \quad w_1 x_1^3 + w_2 x_2^3 = 0 \tag{1.5}$$

Due to the symmetry of the interval $[-1, 1]$, we can assume $x_1 = -x_2$ and $w_1 = w_2$. From (1.2), substituting $w_1 = w_2$:

$$\begin{aligned} 2w_1 = 2 &\implies w_1 = 1 \\ \therefore w_2 &= 1 \end{aligned}$$

From (1.4), substituting the weights and $x_1 = -x_2$:

$$\begin{aligned} x_1^2 + (-x_1)^2 &= \frac{2}{3} \\ \implies 2x_1^2 &= \frac{2}{3} \\ \implies x_1 &= \pm\frac{1}{\sqrt{3}} \end{aligned}$$

We choose $x_1 = -1/\sqrt{3}$ and $x_2 = 1/\sqrt{3}$.

1.2.2 Final 2-Point Formula

Substituting the nodes and weights into (1.1):

$$\int_{-1}^{1} f(x)\,dx \approx f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right)$$

1.3 3-Point Rule

The integral approximation for the 3-point rule is given by:

$$\int_{-1}^{1} f(x)\,dx \approx w_1 f(x_1) + w_2 f(x_2) + w_3 f(x_3)$$

Here we have 6 unknowns (3 nodes, 3 weights), allowing for exactness up to degree $2n - 1 = 5$.

1.3.1 System of Equations

The formula must be exact for $f(x) = 1, x, x^2, x^3, x^4, x^5$:

$$f(x) = 1: \quad \int_{-1}^{1} 1\,dx = 2 = w_1 + w_2 + w_3 \tag{1.6}$$

$$f(x) = x: \quad \int_{-1}^{1} x\,dx = 0 = w_1 x_1 + w_2 x_2 + w_3 x_3 \tag{1.7}$$

$$f(x) = x^2: \quad \int_{-1}^{1} x^2\,dx = \frac{2}{3} = w_1 x_1^2 + w_2 x_2^2 + w_3 x_3^2 \tag{1.8}$$

$$f(x) = x^3: \quad \int_{-1}^{1} x^3\,dx = 0 = w_1 x_1^3 + w_2 x_2^3 + w_3 x_3^3 \tag{1.9}$$

$$f(x) = x^4: \quad \int_{-1}^{1} x^4\,dx = \frac{2}{5} = w_1 x_1^4 + w_2 x_2^4 + w_3 x_3^4 \tag{1.10}$$

$$f(x) = x^5: \quad \int_{-1}^{1} x^5\,dx = 0 = w_1 x_1^5 + w_2 x_2^5 + w_3 x_3^5 \tag{1.11}$$

1.3.2 Solving with Symmetry

By symmetry of the interval, we assume: $x_2 = 0$, $x_3 = -x_1$, and $w_3 = w_1$. This simplifies the system significantly.

Finding the Nodes (x_i)

The even-powered equations simplify:

$$\text{From (1.6):} \quad 2w_1 + w_2 = 2 \tag{1.12}$$

$$\text{From (1.8):} \quad 2w_1 x_1^2 = \frac{2}{3} \tag{1.13}$$

$$\text{From (1.10):} \quad 2w_1 x_1^4 = \frac{2}{5} \tag{1.14}$$

Dividing (1.14) by (1.13) to eliminate w_1:

$$\frac{2w_1 x_1^4}{2w_1 x_1^2} = \frac{2/5}{2/3}$$

$$x_1^2 = \frac{3}{5} \implies x_1 = \pm\sqrt{\frac{3}{5}}$$

Therefore, the nodes are: $x_1 = -\sqrt{\frac{3}{5}}$, $x_2 = 0$, and $x_3 = \sqrt{\frac{3}{5}}$.

Finding the Weights (w_i)

Substitute $x_1^2 = 3/5$ into (1.13):

$$2w_1\left(\frac{3}{5}\right) = \frac{2}{3} \implies w_1 = \frac{5}{9}$$

Since $w_3 = w_1$, we have $w_3 = 5/9$.

Finally, use the simplest equation, $2w_1 + w_2 = 2$, to find w_2:

$$2\left(\frac{5}{9}\right) + w_2 = 2$$
$$\frac{10}{9} + w_2 = 2 \implies w_2 = 2 - \frac{10}{9} = \frac{8}{9}$$

1.3.3 Final 3-Point Formula

Substituting the nodes and weights gives the final formula:

$$\int_{-1}^{1} f(x)\,dx \approx \frac{5}{9} f\left(-\sqrt{\frac{3}{5}}\right) + \frac{8}{9} f(0) + \frac{5}{9} f\left(\sqrt{\frac{3}{5}}\right)$$

1.4 Error Analysis

If $f \in O^{2n}([-1,1])$, then there exists $\xi \in (-1,1)$ such that

$$E_n(f) = \int_{-1}^{1} f(x)\,dx - \sum_{i=1}^{n} w_i f(x_i) = C_n f^{(2n)}(\xi),$$

where the constant C_n is

$$C_n = \frac{2^{2n+1}(n!)^4}{(2n+1)((2n)!)^3}.$$

So the error is proportional to the $2n$-th derivative evaluated at some point in the interval.

Numeric values of C_n (for small n) are:

- $n = 1$: $C_1 = \frac{1}{3} \approx 0.333333$
- $n = 2$: $C_2 \approx 0.0074074074$
- $n = 3$: $C_3 \approx 6.3492 \times 10^{-5}$
- $n = 4$: $C_4 \approx 2.8795 \times 10^{-7}$

1.5 Code for Gaussian Quadrature

```python
import numpy as np
from numpy.linalg import eigh

def gauss_legendre_golub_welsch(n):

    diag_a = np.zeros(n)
    k = np.arange(1, n)
    beta = k**2 / (4 * k**2 - 1)
    diag_b = np.sqrt(beta)
    J = np.diag(diag_a) + np.diag(diag_b, k=1) + np.diag(diag_b, k=-1)
    eigenvalues, eigenvectors = eigh(J)
    nodes = eigenvalues
    mu_0 = 2.0
    weights = mu_0 * (eigenvectors[0, :])**2
    return nodes, weights

n = 3
x, w = gauss_legendre_golub_welsch(n)

print(f"Number of points (n): {n}")
print(f"Nodes (x_i): {x}")
print(f"Weights (w_i): {w}")
```

Listing 1.1: Golub-Welsch Algorithm for Gauss-Legendre Quadrature

n	Nodes (x_i)		Weights (W_i)	
	Exact Form	**Decimal (10 Places)**	**Exact Form**	**Decimal (10 Places)**
1	0	0.0000000000	2	2.0000000000
2	$\pm\frac{1}{\sqrt{3}}$	±0.5773502692	1	1.0000000000
3	0	0.0000000000	$\frac{8}{9}$	0.8888888889
	$\pm\sqrt{\frac{3}{5}}$	±0.7745966692	$\frac{5}{9}$	0.5555555556
4	$\pm\sqrt{\frac{3}{7}-\frac{2}{7}\sqrt{\frac{6}{5}}}$	±0.3399810436	$\frac{18+\sqrt{30}}{36}$	0.6521451549
	$\pm\sqrt{\frac{3}{7}+\frac{2}{7}\sqrt{\frac{6}{5}}}$	±0.8611363116	$\frac{18-\sqrt{30}}{36}$	0.3478548451
5	0	0.0000000000	$\frac{128}{225}$	0.5688888889
	$\pm\frac{1}{3}\sqrt{5-2\sqrt{\frac{10}{7}}}$	±0.5384693101	$\frac{322+13\sqrt{70}}{900}$	0.4786286705
	$\pm\frac{1}{3}\sqrt{5+2\sqrt{\frac{10}{7}}}$	±0.9061798459	$\frac{322-13\sqrt{70}}{900}$	0.2369268851

Table 1.1: Nodes (x_i) and Weights (W_i) for n-Point Gauss–Legendre Rules (Exact vs. Decimal)

Chapter 2

Radau Quadrature

The Gauss-Radau quadrature rule is a variation of Gaussian quadrature where one of the endpoints of the integration interval $[-1, 1]$ is required to be a node. Assuming the left endpoint, $\mathbf{x_1 = -1}$, is fixed:

$$\int_{-1}^{1} f(x)\,dx = w_1 f(-1) + \sum_{i=2}^{n} w_i f(x_i) \tag{2.1}$$

For n total nodes, there are $2n-1$ unknowns ($n-1$ variable nodes, n weights). This structure allows the rule to be exact for polynomials of degree up to $\mathbf{2n-2}$.

2.1 3-Point Rule ($n = 3$)

For $n = 3$, the rule is exact for polynomials up to degree $2n - 2 = 4$. We have five unknown parameters: w_1, w_2, w_3, x_2, x_3. The approximation is:

$$\int_{-1}^{1} f(x)\,dx = w_1 f(-1) + w_2 f(x_2) + w_3 f(x_3) \tag{2.2}$$

2.1.1 System of Governing Equations

We enforce exactness for the basis functions $f(x) = 1, x, x^2, x^3, x^4$:

$$f(x) = 1: \quad \int_{-1}^{1} 1\,dx = 2 \quad \Longrightarrow \quad w_1 + w_2 + w_3 = 2 \tag{2.3}$$

$$f(x) = x: \quad \int_{-1}^{1} x\,dx = 0 \quad \Longrightarrow \quad -w_1 + w_2 x_2 + w_3 x_3 = 0 \tag{2.4}$$

$$f(x) = x^2: \quad \int_{-1}^{1} x^2\,dx = \frac{2}{3} \quad \Longrightarrow \quad w_1 + w_2 x_2^2 + w_3 x_3^2 = \frac{2}{3} \tag{2.5}$$

$$f(x) = x^3: \quad \int_{-1}^{1} x^3\,dx = 0 \quad \Longrightarrow \quad -w_1 + w_2 x_2^3 + w_3 x_3^3 = 0 \tag{2.6}$$

$$f(x) = x^4: \quad \int_{-1}^{1} x^4\,dx = \frac{2}{5} \quad \Longrightarrow \quad w_1 + w_2 x_2^4 + w_3 x_3^4 = \frac{2}{5} \tag{2.7}$$

2.1.2 Solution

The nodes x_2, x_3 are the roots of the polynomial $P_3(x) - P_2(x)$, where $P_k(x)$ is the Legendre polynomial of degree k. The solutions are found to be:

$$x_1 = -1, \quad x_2 = \frac{1-\sqrt{6}}{5}, \quad x_3 = \frac{1+\sqrt{6}}{5}$$

The corresponding weights are:

$$w_1 = \frac{2}{9}, \quad w_2 = \frac{16+\sqrt{6}}{18}, \quad w_3 = \frac{16-\sqrt{6}}{18}$$

2.1.3 Final 3-Point Formula

Substituting the nodes and weights into (2.2) yields the final formula:

$$\int_{-1}^{1} f(x)\,dx \approx \frac{2}{9}f(-1) + \frac{16+\sqrt{6}}{18} f\left(\frac{1-\sqrt{6}}{5}\right) + \frac{16-\sqrt{6}}{18} f\left(\frac{1+\sqrt{6}}{5}\right)$$

2.2 Summary of Nodes and Weights

The table below summarizes the Radau-Left nodes and weights, where $x_1 = -1$.

Table 2.1: Nodes (x_i) and Weights (W_i) for n-Point Radau Quadrature ($x_1 = -1$ fixed)

n	**Fixed Node**	**Variable Nodes (x_i)**	**Weights (w_i)**
2	$x_1 = -1$	$x_2 = \frac{1}{3} \approx 0.3333$	$w_1 = \frac{1}{2} = 0.5$ $w_2 = \frac{3}{2} = 1.5$
3	$x_1 = -1$	$x_2 = \frac{1-\sqrt{6}}{5} \approx -0.2899$ $x_3 = \frac{1+\sqrt{6}}{5} \approx 0.6899$	$w_1 = \frac{2}{9} \approx 0.2222$ $w_2 = \frac{16+\sqrt{6}}{18} \approx 1.0250$ $w_3 = \frac{16-\sqrt{6}}{18} \approx 0.7528$

2.3 Error Analysis

The error (remainder term) for the n-point Radau rule (with $x = -1$ fixed) is a function of the $(2n-1)$-th derivative of $f(x)$.

For $f \in C^{2n-1}[-1, 1]$, the error $R_n[f]$ is:

$$R_n[f] = C_n f^{(2n-1)}(\xi), \quad \xi \in (-1, 1)$$

where C_n is a constant related to the integral of the squared polynomial product over the weighted interval.

2.3.1 Actionable Insight

The crucial detail is that the error is governed by the $f^{(2n-1)}(\xi)$ term, which is why the degree of exactness is **$2n-2$**. If $f(x)$ is a polynomial of degree $2n-2$ or less, its $(2n-1)$-th derivative is zero, ensuring the quadrature is exact.

2.4 Code for Radau Quadrature

The implementation of Radau quadrature relies on the **roots_jacobi** function from `scipy.special`, as Radau nodes are roots of Jacobi polynomials with specific parameters ($\alpha = 0, \beta = 1$ for the $x = -1$ fixed case).

```python
import numpy as np
from scipy.special import roots_jacobi

def calculate_radau_quadrature(N: int, fixed_endpoint: float = -1.0):
    if N < 2:
        raise ValueError("N must be at least 2 for Radau quadrature.")
    if fixed_endpoint not in [-1.0, 1.0]:
        raise ValueError("fixed_endpoint must be -1.0 or 1.0.")

    n_variable = N - 1

    # Determine Jacobi parameters (alpha, beta) based on fixed endpoint
    if fixed_endpoint == -1.0:
        # Radau-Left: Fixed node at -1. Corresponds to Jacobi (0, 1)
        alpha = 0
        beta = 1
    else:
        # Radau-Right: Fixed node at 1. Corresponds to Jacobi (1, 0)
        alpha = 1
        beta = 0

    # Get nodes and weights for the underlying n-1 point Jacobi rule
    t, w_jacobi = roots_jacobi(n_variable, alpha, beta)

    # Transform Jacobi weights to Radau weights for the n-1 variable nodes
    if fixed_endpoint == -1.0:
        # Weights for variable nodes (x_2 to x_n)
```

```
        w_radau = 2.0 * w_jacobi / (1.0 + t)
    else:
        # Weights for variable nodes (x_1 to x_{n-1})
        w_radau = 2.0 * w_jacobi / (1.0 - t)

    # Calculate the weight for the fixed endpoint
    w_fixed = 2.0 - np.sum(w_radau)

    # Combine nodes and weights
    # Insert fixed node and weight at the correct position (0 for -1, end for 1)
    nodes = np.insert(t, 0 if fixed_endpoint == -1.0 else n_variable, fixed_endpoint)
    weights = np.insert(w_radau, 0 if fixed_endpoint == -1.0 else n_variable, w_fixed)

    # Sort nodes and weights for standard presentation
    sort_indices = np.argsort(nodes)
    return nodes[sort_indices], weights[sort_indices]

N_points = 3
nodes_3, weights_3 = calculate_radau_quadrature(N_points, fixed_endpoint=-1.0)
print(f"--- N={N_points} Radau-Left Rule (Fixed x=-1) ---")
print(f"Nodes (x_i):   {nodes_3}")
print(f"Weights (w_i): {weights_3}")
```

Listing 2.1: Code for Radau Quadrature using SciPy

Chapter 3

Lobatto Quadrature Derivation

The Gauss-Lobatto quadrature rule is a form of closed Gaussian quadrature where **both endpoints**, $\mathbf{x_1 = -1}$ and $\mathbf{x_n = 1}$, are included as fixed nodes. The general approximation form is:

$$\int_{-1}^{1} f(x)\,dx = w_1 f(-1) + w_n f(1) + \sum_{i=2}^{n-1} w_i f(x_i) \tag{3.1}$$

With n total nodes, there are $2n-2$ unknown parameters ($n-2$ variable nodes, n weights). This structure ensures the method is exact for polynomials of degree up to $\mathbf{2n-3}$.

3.1 3-Point Rule ($n = 3$)

For $n = 3$, the rule is exact for polynomials up to degree $2n - 3 = 3$. We have four unknown parameters: w_1, w_2, w_3, x_2. The approximation is:

$$\int_{-1}^{1} f(x)\,dx = w_1 f(-1) + w_3 f(1) + w_2 f(x_2) \tag{3.2}$$

3.1.1 System of Governing Equations

We enforce exactness for the basis functions $f(x) = 1, x, x^2, x^3$:

$$f(x) = 1: \quad \int_{-1}^{1} 1\,\mathrm{d}x = 2 \quad \Longrightarrow \quad w_1 + w_2 + w_3 = 2 \tag{3.3}$$

$$f(x) = x: \quad \int_{-1}^{1} x\,\mathrm{d}x = 0 \quad \Longrightarrow \quad -w_1 + w_2 x_2 + w_3 = 0 \tag{3.4}$$

$$f(x) = x^2: \quad \int_{-1}^{1} x^2\,\mathrm{d}x = \frac{2}{3} \quad \Longrightarrow \quad w_1 + w_2 x_2^2 + w_3 = \frac{2}{3} \tag{3.5}$$

$$f(x) = x^3: \quad \int_{-1}^{1} x^3\,\mathrm{d}x = 0 \quad \Longrightarrow \quad -w_1 + w_2 x_2^3 + w_3 = 0 \tag{3.6}$$

3.1.2 Solution

Due to the symmetry of the interval and moments:

1. From (3.4) and (3.6), we deduce that $w_2x_2 = w_2x_2^3$. Since $w_2 \neq 0$ (from (3.3)), we must have $x_2 = x_2^3$, which implies $\mathbf{x_2 = 0}$ or $x_2 = \pm 1$ (but the latter is fixed).
2. Substituting $x_2 = 0$ into (3.5): $w_1 + 0 + w_3 = \frac{2}{3}$.
3. Substituting $w_1 + w_3 = \frac{2}{3}$ into (3.3): $\frac{2}{3} + w_2 = 2 \implies \mathbf{w_2 = \frac{4}{3}}$.
4. Substituting w_2 and x_2 into (3.4) gives $-w_1 + w_3 = 0$, so $\mathbf{w_1 = w_3}$.
5. Since $w_1 + w_3 = \frac{2}{3}$, we have $2w_1 = \frac{2}{3} \implies \mathbf{w_1 = w_3 = \frac{1}{3}}$.

3.1.3 Final 3-Point Formula (Simpson's Rule)

Substituting the nodes and weights into (3.2) yields:

$$\int_{-1}^{1} f(x)\,dx \approx \frac{1}{3}f(-1) + \frac{4}{3}f(0) + \frac{1}{3}f(1) = \frac{1}{3}\left[f(-1) + 4f(0) + f(1)\right]$$

3.2 Summary of Nodes and Weights

Table 3.1: Nodes (x_i) and Weights (W_i) for n-Point Lobatto Quadrature

n	**Fixed Nodes**	**Variable Nodes (x_i)**	**Weights (w_i)**
3	± 1	0	$w_1, w_3 = 1/3 \approx 0.3333$ $w_2 = 4/3 \approx 1.3333$
4	± 1	$x_2, x_3 = \pm\sqrt{\frac{1}{5}} \approx \pm 0.44721$	$w_1, w_4 = 1/6 \approx 0.1667$ $w_2, w_3 = 5/6 \approx 0.8333$
5	± 1	$x_2, x_4 = \pm\sqrt{\frac{3}{7}} \approx \pm 0.65465$ $x_3 = 0$	$w_1, w_5 = 1/10 = 0.1000$ $w_2, w_4 = 49/90 \approx 0.5444$ $w_3 = 32/45 \approx 0.7111$

3.3 Error Analysis

The error (or remainder term), $E_n(f)$, for the n-point Lobatto rule is structurally tied to the $2n - 2$ derivative of the integrand $f(x)$.

3.3.1 Mean-Value Form

For a smooth function $f \in C^{2n-2}([-1,1])$, the error is:

$$E_n(f) = C_n^{(L)} f^{(2n-2)}(\xi), \quad \xi \in (-1,1)$$

where $C_n^{(L)}$ is a constant determined by the specific Lobatto rule, and ξ is some point within the integration interval.

3.3.2 Actionable Insight

The degree of exactness is $\mathbf{2n-3}$. Since the error is proportional to the $(\mathbf{2n-2})$-th derivative, if $f(x)$ is a polynomial of degree $2n-3$ or less, $f^{(2n-2)}(x)$ is zero, and the quadrature yields the exact result.

3.4 Code for Lobatto Quadrature

The Lobatto nodes are the roots of the Legendre polynomial of degree $n-1$, $P_{n-1}(x)$, plus the endpoints ± 1. The variable nodes (x_2 to x_{n-1}) are the roots of the derivative of the Legendre polynomial, $P'_{n-1}(x)$, which are related to the Jacobi polynomials $P_{n-2}^{(1,1)}(x)$. The `scipy.special.roots_jacobi` function handles this efficiently.

```
import numpy as np
from scipy.special import roots_jacobi

def calculate_lobatto_quadrature(N: int):
    # N is the total number of nodes (including -1 and 1).
    if N < 2:
        raise ValueError("N must be at least 2 for Lobatto quadrature.")

    if N == 2:
        # Trivial case: Trapezoidal rule
        nodes = np.array([-1.0, 1.0])
        weights = np.array([1.0, 1.0])
        return nodes, weights

    # Lobatto variable nodes are roots of Jacobi P_{N-2}^{(1, 1)}(x)
    n_variable = N - 2
    alpha = 1
    beta = 1

    # Get the internal nodes (t) and their weights (w_jacobi)
    t, w_jacobi = roots_jacobi(n_variable, alpha, beta)

```

```
    # Calculate the weight for the fixed endpoints (w_endpoints)
    # The general formula for the endpoint weights is w_1 = w_n = 2 / (N*(N-1))
    w_endpoints = 2.0 / (N * (N - 1))

    # Calculate the weights for the internal nodes
    # w_i = (2 / (N*(N-1))) * (1 - t_i^2) * P_{N-2}^{(1, 1)}(t_i)
    # The weights returned by roots_jacobi for (1, 1) are w_jacobi,
    # which need scaling to get the actual w_internal
    w_internal = w_jacobi * (2.0 / ((N * (N-1)))) # Apply the correct scaling

    # Combine nodes and weights: [-1.0, t_2, ..., t_{n-1}, 1.0]
    nodes = np.concatenate(([-1.0], t, [1.0]))
    weights = np.concatenate(([w_endpoints], w_internal, [w_endpoints]))

    return nodes, weights

N_points = 3 # This will calculate the N=3 rule (3 nodes: -1, 0, 1)
nodes, weights = calculate_lobatto_quadrature(N_points)

print(f"--- N={N_points} Lobatto Rule (Fixed x=-1, x=1) ---")
print(f"Nodes (x_i):   {nodes}")
print(f"Weights (w_i): {weights}")
```

Listing 3.1: Code for Lobatto Quadrature using SciPy

Chapter 4

Gauss–Hermite Quadrature Derivation

The Gauss-Hermite quadrature rule is a powerful numerical technique for approximating integrals over the infinite interval $(-\infty, \infty)$ where the integrand includes a standard Gaussian weighting function, $w(x) = e^{-x^2}$.

$$\int_{-\infty}^{\infty} e^{-x^2} f(x)\, dx \approx \sum_{i=1}^{n} w_i f(x_i)$$

The nodes (x_i) are the roots of the Hermite polynomial $H_n(x)$, and the weights (w_i) are determined such that the rule is exact for polynomials $f(x)$ up to degree $2n-1$. The Gauss-Hermite quadrature formula is:

$$\int_{-\infty}^{\infty} f(x) e^{-x^2} dx = \sum_{k=1}^{n} A_k f(x_k)$$

4.1 Transformation Method

For the integral transformation:

$$\int_a^b f(x)dx \to \int_{-1}^{1} F(t)dt$$

Using the hyperbolic tangent transformation:

$$x = \frac{b-a}{2}\tanh(t) + \frac{a+b}{2}$$

The derivative is:

$$\frac{dx}{dt} = \frac{b-a}{2}\operatorname{sech}^2(t)$$

Therefore, the transformed function becomes:

$$F(t) = f(x(t))\frac{dx}{dt} = f\left(\frac{b-a}{2}\tanh(t) + \frac{a+b}{2}\right) \cdot \frac{b-a}{2}\operatorname{sech}^2(t)$$

4.2 2-Point Rule ($n = 2$)

For the $n = 2$ rule, we determine four unknowns (w_1, w_2, x_1, x_2) by enforcing exactness for polynomials of degree up to $2n - 1 = 3$. We use the basis functions $1, x, x^2, x^3$.

4.2.1 System of Governing Equations

We use the known exact values for the moments $\int_{-\infty}^{\infty} x^k e^{-x^2} dx$:

$$f(x) = 1: \quad \int_{-\infty}^{\infty} 1e^{-x^2}\, dx = \sqrt{\pi} \quad \Longrightarrow \quad w_1 + w_2 = \sqrt{\pi} \tag{4.1}$$

$$f(x) = x: \quad \int_{-\infty}^{\infty} xe^{-x^2}\, dx = 0 \quad \Longrightarrow \quad w_1 x_1 + w_2 x_2 = 0 \tag{4.2}$$

$$f(x) = x^2: \quad \int_{-\infty}^{\infty} x^2 e^{-x^2}\, dx = \frac{\sqrt{\pi}}{2} \quad \Longrightarrow \quad w_1 x_1^2 + w_2 x_2^2 = \frac{\sqrt{\pi}}{2} \tag{4.3}$$

Due to the symmetry of the Gaussian weight function, we can assume $x_1 = -x_2$ and $w_1 = w_2$.

From (4.1), substituting $w_1 = w_2$:

$$2w_1 = \sqrt{\pi} \implies w_1 = \frac{\sqrt{\pi}}{2}$$
$$\therefore w_2 = \frac{\sqrt{\pi}}{2}$$

From (4.3), substituting the weights and $x_1 = -x_2$:

$$w_1 x_1^2 + w_1 x_1^2 = \frac{\sqrt{\pi}}{2}$$
$$2w_1 x_1^2 = \frac{\sqrt{\pi}}{2}$$
$$2\left(\frac{\sqrt{\pi}}{2}\right) x_1^2 = \frac{\sqrt{\pi}}{2}$$
$$\implies x_1^2 = \frac{1}{2} \implies x_1 = \pm\frac{1}{\sqrt{2}}$$

We choose $x_1 = -1/\sqrt{2}$ and $x_2 = 1/\sqrt{2}$.

4.2.2 Final 2-Point Formula

Substituting the nodes and weights into the general approximation formula:

$$\int_{-\infty}^{\infty} e^{-x^2} f(x)\, dx \approx \frac{\sqrt{\pi}}{2} f\left(-\frac{1}{\sqrt{2}}\right) + \frac{\sqrt{\pi}}{2} f\left(\frac{1}{\sqrt{2}}\right)$$

4.3 3-Point Rule ($n = 3$)

The 3-point rule involves 6 unknowns, allowing for exactness up to degree $2n - 1 = 5$. The integral approximation is:

$$\int_{-\infty}^{\infty} e^{-x^2} f(x)\, dx \approx w_1 f(x_1) + w_2 f(x_2) + w_3 f(x_3)$$

4.3.1 System of Governing Equations

The formula must be exact for $f(x) = 1, x, x^2, x^3, x^4, x^5$:

$$f(x) = 1: \quad w_1 + w_2 + w_3 = \sqrt{\pi} \tag{4.4}$$

$$f(x) = x: \quad w_1 x_1 + w_2 x_2 + w_3 x_3 = 0 \tag{4.5}$$

$$f(x) = x^2: \quad w_1 x_1^2 + w_2 x_2^2 + w_3 x_3^2 = \frac{\sqrt{\pi}}{2} \tag{4.6}$$

$$f(x) = x^4: \quad w_1 x_1^4 + w_2 x_2^4 + w_3 x_3^4 = \frac{3\sqrt{\pi}}{4} \tag{4.7}$$

(Equations for $f(x) = x^3$ and $f(x) = x^5$ also equal zero and are redundant under symmetry.)

4.3.2 Solving with Symmetry

We assume symmetry: $\mathbf{x_2 = 0}$, $\mathbf{x_3 = -x_1}$, and $\mathbf{w_3 = w_1}$.

Finding the Nodes (x_i)

The even-powered equations simplify using $x_2 = 0$ and $w_1 = w_3$:

$$\text{From (4.6):} \quad 2w_1 x_1^2 = \frac{\sqrt{\pi}}{2} \tag{4.8}$$

$$\text{From (4.7):} \quad 2w_1 x_1^4 = \frac{3\sqrt{\pi}}{4} \tag{4.9}$$

Dividing (4.9) by (4.8) to find x_1^2:

$$\frac{2w_1 x_1^4}{2w_1 x_1^2} = \frac{3\sqrt{\pi}/4}{\sqrt{\pi}/2}$$

$$x_1^2 = \frac{3}{4} \cdot 2 = \frac{3}{2}$$

$$\implies x_1 = \pm\sqrt{\frac{3}{2}}$$

The nodes are: $\mathbf{x_1 = -\sqrt{3/2}}$, $\mathbf{x_2 = 0}$, and $\mathbf{x_3 = \sqrt{3/2}}$.

Finding the Weights (w_i)

Substitute $x_1^2 = 3/2$ into (4.8):

$$2w_1\left(\frac{3}{2}\right) = \frac{\sqrt{\pi}}{2}$$
$$3w_1 = \frac{\sqrt{\pi}}{2} \implies w_1 = \frac{\sqrt{\pi}}{6}$$

Since $w_3 = w_1$, we have $\mathbf{w_3} = \sqrt{\pi}/\mathbf{6}$.

Finally, use (4.4) to find w_2:

$$2w_1 + w_2 = \sqrt{\pi}$$
$$2\left(\frac{\sqrt{\pi}}{6}\right) + w_2 = \sqrt{\pi}$$
$$w_2 = \sqrt{\pi} - \frac{\sqrt{\pi}}{3} = \frac{2\sqrt{\pi}}{3}$$

The weights are: $\mathbf{w_1} = \sqrt{\pi}/\mathbf{6}$, $\mathbf{w_2} = \mathbf{2}\sqrt{\pi}/\mathbf{3}$, and $\mathbf{w_3} = \sqrt{\pi}/\mathbf{6}$.

4.3.3 Final 3-Point Formula

$$\int_{-\infty}^{\infty} e^{-x^2} f(x)\,dx \approx \frac{\sqrt{\pi}}{6} f\left(-\sqrt{\frac{3}{2}}\right) + \frac{2\sqrt{\pi}}{3} f(0) + \frac{\sqrt{\pi}}{6} f\left(\sqrt{\frac{3}{2}}\right)$$

4.4 Summary of Nodes and Weights

n	**Nodes (x_i)**		**Weights (W_i)**	
	Exact Form	**Decimal (5 Places)**	**Exact Form**	**Decimal (5 Places)**
1	0	0.00000	$\sqrt{\pi}$	1.77245
2	$\pm\frac{1}{\sqrt{2}}$	$\pm$0.70711	$\frac{\sqrt{\pi}}{2}$	0.88623
3	0	0.00000	$\frac{2\sqrt{\pi}}{3}$	1.18164
	$\pm\sqrt{\frac{3}{2}}$	$\pm$1.22474	$\frac{\sqrt{\pi}}{6}$	0.29541

Table 4.1: Nodes (x_i) and Weights (W_i) for n-Point Gauss–Hermite Rules

4.5 Error Analysis

If $f \in O^{2n}(\mathbb{R})$, the error (remainder term) for the n-point Gauss-Hermite rule is proportional to the $2n$-th derivative evaluated at some point ξ:

$$R_n[f] = \int_{-\infty}^{\infty} e^{-x^2} f(x)\, dx - \sum_{i=1}^{n} w_i f(x_i) = C_n f^{(2n)}(\xi)$$

where the constant C_n is:

$$C_n = \frac{n!\sqrt{\pi}}{2^n(2n)!}$$

The rapid decrease of C_n with increasing n illustrates the high convergence rate of the Gauss-Hermite method.

4.6 Code for Gauss-Hermite Quadrature

The most robust way to find higher-order nodes and weights is using the relationship with the eigenvalues of the tridiagonal matrix defined by the recurrence relation for Hermite polynomials (the Golub-Welsch algorithm).

```
import numpy as np
from numpy.polynomial.hermite import hermgauss

def calculate_gauss_hermite(N):
    # hermgauss(N) returns the nodes and weights for the
    # integral $\int_{-\infty}^{\infty} e^{-x^2} f(x) dx$
    if N < 1:
        raise ValueError("N must be at least 1.")
    nodes, weights = hermgauss(N)
    return nodes, weights

N_points = 3
nodes, weights = calculate_gauss_hermite(N_points)

print(f"Number of points (n): {N_points}")
# Nodes for n=3 should be: +/- 1.22474... and 0
print(f"Nodes (x_i): {nodes}")
# Weights for n=3 should be: [0.29541, 1.18164, 0.29541] * sqrt
    (pi)
print(f"Weights (w_i): {weights}")
```

Listing 4.1: Python Code for Gauss-Hermite Quadrature (using numpy)

Chapter 5

Gauss–Laguerre Quadrature Derivation

The Gauss-Laguerre quadrature rule is used to approximate integrals over the semi-infinite interval $[0, \infty)$ involving the exponential weighting function, $w(x) = e^{-x}$:

$$\int_0^\infty e^{-x} f(x)\, dx \approx \sum_{i=1}^{n} w_i f(x_i)$$

Here, n is the number of nodes. The rule is exact when the function $f(x)$ is a polynomial of degree up to $2n - 1$.

The standard Gauss-Laguerre quadrature integrates functions of the form:

$$\int_0^\infty f(x) e^{-x} dx$$

5.1 Transformation Method

To transform a finite interval integral:

$$\int_a^b f(x) dx \to \int_0^\infty F(t) e^{-t} dt$$

Use the rational transformation:

$$x = a + (b - a)\frac{t}{1 + t}$$

The corresponding derivative is:

$$\frac{dx}{dt} = \frac{b - a}{(1 + t)^2}$$

The transformed function becomes:

$$F(t) = f(x(t))\frac{dx}{dt} = f\left(a + (b - a)\frac{t}{1 + t}\right) \cdot \frac{b - a}{(1 + t)^2}$$

For the Gauss-Laguerre application, multiply by e^t:

$$F(t) = f\left(a + (b-a)\frac{t}{1+t}\right) \cdot \frac{b-a}{(1+t)^2} \cdot e^t$$

5.2 1-Point Rule ($n = 1$)

For the $n = 1$ rule, we determine two unknowns (w_1, x_1) by enforcing exactness for polynomials of degree up to $2n - 1 = 1$. We use the basis functions 1 and x.

5.2.1 System of Governing Equations

We use the known results for the Gamma function: $\int_0^\infty x^k e^{-x} dx = k!$.

$$f(x) = 1: \quad \int_0^\infty 1e^{-x}\,dx = 0! = 1 \qquad \Longrightarrow \quad w_1 \qquad = 1 \tag{5.1}$$

$$f(x) = x: \quad \int_0^\infty xe^{-x}\,dx = 1! = 1 \qquad \Longrightarrow \quad w_1 x_1 \qquad = 1 \tag{5.2}$$

5.2.2 Solution

From (5.1), we get $\mathbf{w_1 = 1}$.

Substituting $w_1 = 1$ into (5.2):

$$(1)x_1 = 1 \implies \mathbf{x_1 = 1}$$

5.2.3 Final 1-Point Formula

$$\int_0^\infty e^{-x} f(x)\,dx \approx f(1)$$

5.3 2-Point Rule ($n = 2$)

For the $n = 2$ rule, we determine four unknowns (w_1, w_2, x_1, x_2), enforcing exactness for polynomials up to degree $2n - 1 = 3$.

5.3.1 System of Governing Equations

We enforce exactness for $f(x) = 1, x, x^2, x^3$:

$$f(x) = 1: \quad \int_0^\infty e^{-x} dx = 1 \quad \Longrightarrow \quad w_1 + w_2 \quad = 1 \tag{5.3}$$

$$f(x) = x: \quad \int_0^\infty x e^{-x} dx = 1 \quad \Longrightarrow \quad w_1 x_1 + w_2 x_2 \quad = 1 \tag{5.4}$$

$$f(x) = x^2: \quad \int_0^\infty x^2 e^{-x} dx = 2 \quad \Longrightarrow \quad w_1 x_1^2 + w_2 x_2^2 \quad = 2 \tag{5.5}$$

$$f(x) = x^3: \quad \int_0^\infty x^3 e^{-x} dx = 6 \quad \Longrightarrow \quad w_1 x_1^3 + w_2 x_2^3 \quad = 6 \tag{5.6}$$

5.3.2 Solution

The solution to this non-linear system corresponds to the roots of $L_2(x) = x^2 - 4x + 2 = 0$.

Finding the Nodes (x_i)

The roots of $L_2(x)$ are found using the quadratic formula:

$$x = \frac{-(-4) \pm \sqrt{(-4)^2 - 4(1)(2)}}{2(1)} = \frac{4 \pm \sqrt{16 - 8}}{2} = \frac{4 \pm \sqrt{8}}{2} = 2 \pm \sqrt{2}$$

The nodes are: $\mathbf{x_1 = 2 - \sqrt{2}}$ and $\mathbf{x_2 = 2 + \sqrt{2}}$.

Finding the Weights (w_i)

Using the property that the weights are given by $w_i = -\frac{1}{L_n'(x_i) L_{n-1}(x_i)}$ (or by solving the system):

$$\mathbf{w_1} = \frac{\mathbf{2 + \sqrt{2}}}{\mathbf{4}} \quad \text{and} \quad \mathbf{w_2} = \frac{\mathbf{2 - \sqrt{2}}}{\mathbf{4}}$$

(Note: The weights w_i and nodes x_i are assigned inversely due to the labeling convention for Laguerre polynomials, where x_1 is the smallest root. Here, x_1 corresponds to the larger weight $\frac{2+\sqrt{2}}{4} \approx 0.8536$.)

5.3.3 Final 2-Point Formula

$$\int_0^\infty e^{-x} f(x)\, dx \approx \frac{2 + \sqrt{2}}{4} f(2 - \sqrt{2}) + \frac{2 - \sqrt{2}}{4} f(2 + \sqrt{2})$$

5.4 3-Point Rule ($n = 3$)

The 3-point rule requires 6 equations for exactness up to degree $2n - 1 = 5$. The moments are:

$$\int_0^\infty x^k e^{-x} dx = k!$$

The system of governing equations for the 6 unknowns $(w_1, w_2, w_3, x_1, x_2, x_3)$ is:

$$w_1 + w_2 + w_3 = 1 \tag{5.7}$$
$$w_1 x_1 + w_2 x_2 + w_3 x_3 = 1 \tag{5.8}$$
$$w_1 x_1^2 + w_2 x_2^2 + w_3 x_3^2 = 2 \tag{5.9}$$
$$w_1 x_1^3 + w_2 x_2^3 + w_3 x_3^3 = 6 \tag{5.10}$$
$$w_1 x_1^4 + w_2 x_2^4 + w_3 x_3^4 = 24 \tag{5.11}$$
$$w_1 x_1^5 + w_2 x_2^5 + w_3 x_3^5 = 120 \tag{5.12}$$

The nodes (x_i) are the roots of the Laguerre polynomial $L_3(x) = -x^3 + 9x^2 - 18x + 6 = 0$.

5.5 Summary of Nodes and Weights

The table below provides the exact and high-precision decimal values for the nodes and weights for the first few Gauss-Laguerre rules.

Table 5.1: Nodes (x_i) and Weights (W_i) for n-Point Gauss–Laguerre Rules (Exact vs. Decimal)

# of nodes (n)	**Nodes (x_i)**	**Weights (w_i)**
1	$x_1 = 1$	$w_1 = 1$
2	$x_1 = 2 - \sqrt{2} \approx 0.585786$	$w_1 = \frac{2+\sqrt{2}}{4} \approx 0.853553$
	$x_2 = 2 + \sqrt{2} \approx 3.414214$	$w_2 = \frac{2-\sqrt{2}}{4} \approx 0.146447$
3	$x_1 \approx 0.41577456$	$w_1 \approx 0.71109301$
	$x_2 \approx 2.29428036$	$w_2 \approx 0.27851773$
	$x_3 \approx 6.28994508$	$w_3 \approx 0.01038926$

5.6 Code for Gaussian Laguerre

The `laggauss` function from NumPy provides a stable way to calculate the nodes and weights using numerical methods based on the properties of Laguerre polynomials.

```
import numpy as np
from numpy.polynomial.laguerre import laggauss

def calculate_gauss_laguerre(N):
    if N < 1:
        raise ValueError("N must be at least 1 for Gauss-Laguerre quadrature.")
    # laggauss(N) returns the nodes and weights for the
    # integral $\int_{0}^{\infty} e^{-x} f(x) dx$
    nodes, weights = laggauss(N)
    return nodes, weights

N_points = 3
nodes, weights = calculate_gauss_laguerre(N_points)

print(f"Number of points (n): {N_points}")
print(f"Nodes (x_i): {nodes}")
print(f"Weights (w_i): {weights}")
```

Listing 5.1: Python Code for Gauss-Laguerre Quadrature

5.7 Error Analysis

The error $R_n[f]$ for the n-point Gauss-Laguerre rule provides a measure of its accuracy.

For a sufficiently smooth function f (i.e., $f \in O^{2n}([0,\infty))$), the error is given by:

$$R_n[f] = \int_0^\infty e^{-x} f(x)\, dx - \sum_{i=1}^{n} w_i f(x_i) = C_n f^{(2n)}(\xi)$$

where $\xi \in (0,\infty)$.

The constant C_n for the Laguerre rule is:

$$C_n = \frac{(n!)^2}{(2n)!}$$

5.7.1 Actionable Insight

Similar to other Gaussian methods, the error is proportional to the **$2n$-th derivative** of the function $f(x)$. This means if $f(x)$ is simple, even a small n provides extremely high accuracy. If $f(x)$ is a polynomial of degree $2n-1$ or less, $R_n[f] = 0$, and the result is exact.

Chapter 6

Gauss–Chebyshev Quadrature Derivation

The Gauss-Chebyshev quadrature rule is designed for integrals over the canonical interval $[-1, 1]$ with the specific weighting function $w(x) = \frac{1}{\sqrt{1-x^2}}$:

$$\int_{-1}^{1} \frac{f(x)}{\sqrt{1-x^2}}\,dx \approx \sum_{i=1}^{n} w_i f(x_i)$$

This method is notable because the weights w_i are all equal for any n, and the nodes x_i are the roots of the Chebyshev polynomial of the first kind, $T_n(x)$. The rule is exact for polynomials $f(x)$ up to degree $2n-1$.

6.1 1-Point Rule ($n = 1$)

For $n = 1$, we determine two unknowns (w_1, x_1) by enforcing exactness for polynomials up to degree $2n - 1 = 1$.

6.1.1 System of Governing Equations

$$f(x) = 1: \quad \int_{-1}^{1} \frac{1}{\sqrt{1-x^2}}\,dx = \pi \qquad \Longrightarrow \quad w_1 \qquad = \pi \tag{6.1}$$

$$f(x) = x: \quad \int_{-1}^{1} \frac{x}{\sqrt{1-x^2}}\,dx = 0 \qquad \Longrightarrow \quad w_1 x_1 \qquad = 0 \tag{6.2}$$

6.1.2 Solution

From (6.1), $\mathbf{w_1} = \pi$. Substituting $w_1 = \pi$ into (6.2):

$$\pi x_1 = 0 \implies \mathbf{x_1 = 0}$$

6.1.3 Final 1-Point Formula

$$\int_{-1}^{1} \frac{f(x)}{\sqrt{1-x^2}}\,dx \approx \pi f(0)$$

6.2 2-Point Rule ($n = 2$)

For $n = 2$, we have 4 unknowns, enforcing exactness for polynomials up to degree $2n - 1 = 3$.

6.2.1 System of Governing Equations

We enforce exactness for $f(x) = 1, x, x^2$:

$$f(x) = 1: \quad \int_{-1}^{1} \frac{1}{\sqrt{1-x^2}}\,dx = \pi \quad \Longrightarrow \quad w_1 + w_2 = \pi \tag{6.3}$$

$$f(x) = x: \quad \int_{-1}^{1} \frac{x}{\sqrt{1-x^2}}\,dx = 0 \quad \Longrightarrow \quad w_1 x_1 + w_2 x_2 = 0 \tag{6.4}$$

$$f(x) = x^2: \quad \int_{-1}^{1} \frac{x^2}{\sqrt{1-x^2}}\,dx = \frac{\pi}{2} \quad \Longrightarrow \quad w_1 x_1^2 + w_2 x_2^2 = \frac{\pi}{2} \tag{6.5}$$

6.2.2 Solution via Symmetry

Due to symmetry of the interval and weight function, we assume $\mathbf{x_1 = -x_2}$ and $\mathbf{w_1 = w_2}$.

From (6.3):

$$2w_1 = \pi \implies \mathbf{w_1 = w_2 = \frac{\pi}{2}}$$

From (6.5), substituting the weights and $x_1 = -x_2$:

$$\begin{aligned} w_1 x_1^2 + w_1 x_1^2 &= \frac{\pi}{2} \\ 2w_1 x_1^2 &= \frac{\pi}{2} \\ 2\left(\frac{\pi}{2}\right) x_1^2 &= \frac{\pi}{2} \\ \implies \pi x_1^2 &= \frac{\pi}{2} \\ \implies x_1^2 = \frac{1}{2} &\implies x_1 = \pm\frac{1}{\sqrt{2}} \end{aligned}$$

The nodes are: $\mathbf{x_1 = -1/\sqrt{2}}$ and $\mathbf{x_2 = 1/\sqrt{2}}$.

6.2.3 Final 2-Point Formula

$$\int_{-1}^{1} \frac{f(x)}{\sqrt{1-x^2}}\,dx \approx \frac{\pi}{2} f\left(-\frac{1}{\sqrt{2}}\right) + \frac{\pi}{2} f\left(\frac{1}{\sqrt{2}}\right)$$

6.3 3-Point Rule ($n = 3$)

The 3-point rule requires 6 equations for exactness up to degree $2n - 1 = 5$. The integral moments are required:

$$\int_{-1}^{1} \frac{x^k}{\sqrt{1-x^2}} dx$$

The nodes are the roots of the Chebyshev polynomial $T_3(x) = 4x^3 - 3x$, which are 0 and $\pm\frac{\sqrt{3}}{2}$.

6.3.1 Final 3-Point Formula (Corrected)

For $n = 3$ Gauss-Chebyshev Quadrature, the nodes (x_i) and weights (w_i) are given by the general formulas:

$$x_i = \cos\left(\frac{(2i-1)\pi}{2n}\right), \quad w_i = \frac{\pi}{n}$$

For $n = 3$, the weights are $\mathbf{w_1} = \mathbf{w_2} = \mathbf{w_3} = \frac{\pi}{3}$. The nodes are:

$$x_1 = \cos\left(\frac{\pi}{6}\right) = \frac{\sqrt{3}}{2}$$
$$x_2 = \cos\left(\frac{3\pi}{6}\right) = \cos\left(\frac{\pi}{2}\right) = 0$$
$$x_3 = \cos\left(\frac{5\pi}{6}\right) = -\frac{\sqrt{3}}{2}$$

The final formula is:

$$\int_{-1}^{1} \frac{f(x)}{\sqrt{1-x^2}}\,dx \approx \frac{\pi}{3} f\left(-\frac{\sqrt{3}}{2}\right) + \frac{\pi}{3} f(0) + \frac{\pi}{3} f\left(\frac{\sqrt{3}}{2}\right)$$

6.4 Summary of Nodes and Weights

6.5 Code for Gaussian Quadrature

The `chebgauss` function in NumPy can generate the nodes and weights using numerical stability methods based on the properties of Chebyshev polynomials.

Table 6.1: Nodes (x_i) and Weights (W_i) for n-Point Gauss–Chebyshev Rules

# of nodes (n)	Nodes (x_i)	Weights (w_i)
1	0	π
2	$\pm\frac{1}{\sqrt{2}}$	$\frac{\pi}{2}$
3	0	$\frac{\pi}{3}$
	$\pm\frac{\sqrt{3}}{2}$	$\frac{\pi}{3}$

```python
import numpy as np
from numpy.polynomial.chebyshev import chebgauss

def calculate_gauss_chebyshev(N):
    if N < 1:
        raise ValueError("N must be at least 1 for Gauss-Chebyshev quadrature.")
    # chebgauss(N) returns the nodes and weights for the
    # integral $\int_{-1}^{1} \frac{f(x)}{\sqrt{1-x^2}} dx$
    nodes, weights = chebgauss(N)
    return nodes, weights

N_points = 3
nodes, weights = calculate_gauss_chebyshev(N_points)

print(f"Number of points (n): {N_points}")
# Nodes for n=3 should be: +/- 0.866025... (sqrt(3)/2) and 0.
print(f"Nodes (x_i): {nodes}")
# Weights for n=3 should be: [1.047197..., 1.047197..., 1.047197...] (pi/3)
print(f"Weights (w_i): {weights}")
```

Listing 6.1: Python Code for Gauss-Chebyshev Quadrature

6.6 Error Analysis

The error term $R_n[f]$ quantifies the accuracy of the n-point Gauss-Chebyshev rule (first kind).

For a sufficiently smooth function f (i.e., $f \in O^{2n}([-1,1])$), the error is given by:

$$R_n[f] = \frac{\pi}{2^{2n-1}(2n)!} f^{(2n)}(\xi)$$

where ξ is some point in $(-1, 1)$.

6.6.1 Actionable Insight

The error is proportional to the $2n$-th derivative of $f(x)$, indicating rapid convergence for smooth functions. Crucially, if $f(x)$ is a polynomial of degree $2n-1$ or less, the rule is exact ($R_n[f] = 0$), making it the perfect tool for integrating polynomial-like functions with this specific weight.

6.7 Generalized Gaussian Quadrature Rules

The integration problem can be expressed in a slightly more general way by introducing a positive weight function ω into the integrand, and allowing an interval other than $[-1, 1]$. That is, the problem is to calculate:

$$\int_a^b \omega(x) f(x)\,\mathrm{d}x$$

For some choices of **a**, **b**, and ω that lead to other Quadrature rules. Some of these are tabulated below.

Interval	$\omega(x)$	**Orthogonal polynomials**	**Quadrature Rule**
$[-1, 1]$	1	Legendre polynomials	Gauss-Legendre quadrature
$(-1, 1)$	$(1-x)^\alpha(1+x)^\beta, \alpha, \beta > -1$	Jacobi polynomials	Gauss-Jacobi quadrature
$(-1, 1)$	$\frac{1}{\sqrt{1-x^2}}$	Chebyshev polynomials (first kind)	Chebyshev-Gauss quadrature
$[-1, 1]$	$\sqrt{1-x^2}$	Chebyshev polynomials (second kind)	Chebyshev-Gauss quadrature
$[0, \infty)$	e^{-x}	Laguerre polynomials	Gauss-Laguerre quadrature
$[0, \infty)$	$x^\alpha e^{-x}, \alpha > -1$	Generalized Laguerre polynomials	Gauss-Laguerre quadrature
$(-\infty, \infty)$	e^{-x^2}	Hermite polynomials	Gauss-Hermite quadrature

Chapter 7

Applications of Gaussian Quadrature

7.1 Introduction

In this chapter, we focus on deriving **shape functions based on Lagrange interpolation polynomials** and using them to **transform any physical line or region to a standard reference element**(e.g. the interval $[-1, 1]$ in 1D or the square $[-1, 1] \times [-1, 1]$ in 2D). This transformation allows us to perform interpolation, differentiation, and integration uniformly across elements of various shapes. In finite element analysis, such mappings simplify computations and ensure consistency between physical and reference domains.

We consider three types of shape functions:

- 1D **Linear** shape functions (L_i)
- 1D **Quadratic** shape functions (Q_i)
- 2D **Bilinear** shape functions (N_i)

Each is derived using the same Lagrange interpolation approach, starting from nodal coordinates and values.

7.2 General Lagrange Interpolation (arbitrary n)

For n nodes at $x_1, \ldots, x_n$ with values $u_1, \ldots, u_n$, the Lagrange interpolant is:

$$u_h(x) = \sum_{i=1}^{n} \ell_i(x) u_i, \tag{7.1}$$

where the basis (Lagrange) functions are defined as

$$\ell_i(x) = \prod_{\substack{j=1 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}, \qquad i = 1, \ldots, n. \tag{7.2}$$

Properties (general)

- **Nodal property:** $\ell_i(x_j) = \delta_{ij}$.
- **Partition of unity:** $\sum_{i=1}^{n} \ell_i(x) = 1$.
- **Completeness:** exactly reproduces polynomials up to degree $n-1$.
- **Coordinate interpolation:** $x = \sum_i x_i \ell_i(x)$ when the ℓ_i are constructed from nodal coordinates.
- **Derivative sum:** $\sum_i d\ell_i/dx = 0$.

7.3 1D Linear Shape Functions ($n = 2$, denote by L_i)

Nodes: $x_1 = a$, $x_2 = b$. The two Lagrange functions (linear) are

$$L_1(x) = \frac{x - x_2}{x_1 - x_2} = \frac{b - x}{b - a}, \tag{7.3}$$

$$L_2(x) = \frac{x - x_1}{x_2 - x_1} = \frac{x - a}{b - a}. \tag{7.4}$$

Reference forms (with $\xi \in [-1, 1]$ and $x = \frac{a+b}{2} + \frac{b-a}{2}\xi$):

$$L_1(\xi) = \frac{1 - \xi}{2}, \qquad L_2(\xi) = \frac{1 + \xi}{2}. \tag{7.5}$$

Properties (linear)

- Nodal property: $L_i(x_j) = \delta_{ij}$.
- Partition of unity: $L_1 + L_2 = 1$.
- Linear completeness: reproduces constants and linear functions exactly.
- Coordinate interpolation: $x = x_1 L_1 + x_2 L_2$.
- Derivatives: $\dfrac{dL_1}{dx} = -\dfrac{1}{b-a}$, $\dfrac{dL_2}{dx} = \dfrac{1}{b-a}$ and sum is zero.



Figure 7.1: Reference to physical mapping for 1D linear element.

7.4 1D Quadratic Shape Functions ($n = 3$, denote by Q_i)

Nodes: $x_1 = a$, $x_2 = \frac{a+b}{2}$ (midpoint), $x_3 = b$. The quadratic Lagrange functions are

$$Q_1(x) = \frac{(x - x_2)(x - x_3)}{(x_1 - x_2)(x_1 - x_3)}, \tag{7.6}$$

$$Q_2(x) = \frac{(x - x_1)(x - x_3)}{(x_2 - x_1)(x_2 - x_3)}, \tag{7.7}$$

$$Q_3(x) = \frac{(x - x_1)(x - x_2)}{(x_3 - x_1)(x_3 - x_2)}. \tag{7.8}$$

After algebraic simplification (using $x_2 = \frac{a+b}{2}$):

$$Q_1(x) = \frac{(x - \frac{a+b}{2})(x - b)}{0.5(b - a)^2}, \tag{7.9}$$

$$Q_2(x) = -\frac{2(x - a)(x - b)}{(b - a)^2}, \tag{7.10}$$

$$Q_3(x) = \frac{(x - a)(x - \frac{a+b}{2})}{0.5(b - a)^2}. \tag{7.11}$$

In reference coordinate $\xi \in [-1, 1]$ (nodes at $\xi_1 = -1, \xi_2 = 0, \xi_3 = 1$) the compact forms are

$$Q_1(\xi) = \tfrac{1}{2}\xi(\xi - 1), \tag{7.12}$$

$$Q_2(\xi) = 1 - \xi^2, \tag{7.13}$$

$$Q_3(\xi) = \tfrac{1}{2}\xi(\xi + 1). \tag{7.14}$$

Properties (quadratic)

- Nodal property: $Q_i(\xi_j) = \delta_{ij}$ for $\xi_j = -1, 0, 1$.
- Partition of unity: $Q_1 + Q_2 + Q_3 = 1$.
- Quadratic completeness: reproduces polynomials up to degree 2 exactly.
- Coordinate interpolation: $x = \sum_{i=1}^{3} x_i Q_i(x)$.
- Derivative relations: $\sum_{i=1}^{3} dQ_i/d\xi = 0$ in reference; use chain rule for d/dx.

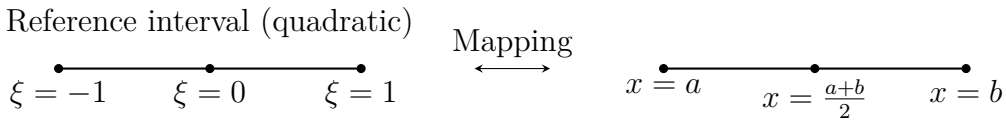


Figure 7.2: Reference to physical mapping for 1D quadratic element (mid-node at $\xi = 0$).

7.5 2D Bilinear Shape Functions ($n = 4$, denote by N_i)

The derivation is performed in the **reference element** (a square) where $\xi, \eta \in [-1, 1]$. The four nodes are numbered counter-clockwise:

- Node 1: $(\xi_1, \eta_1) = (-1, -1)$
- Node 2: $(\xi_2, \eta_2) = (1, -1)$
- Node 3: $(\xi_3, \eta_3) = (1, 1)$
- Node 4: $(\xi_4, \eta_4) = (-1, 1)$

Product of 1D Shape Functions

The 2D bilinear functions are formed by multiplying 1D linear Lagrange functions:

$$N_i(\xi, \eta) = \frac{1}{4}(1 + \xi\xi_i)(1 + \eta\eta_i), \tag{7.15}$$

where (ξ_i, η_i) are nodal coordinates.

Bilinear Shape Functions (N_i)

$$N_1(\xi, \eta) = \frac{1}{4}(1 - \xi)(1 - \eta), \tag{7.16}$$

$$N_2(\xi, \eta) = \frac{1}{4}(1 + \xi)(1 - \eta), \tag{7.17}$$

$$N_3(\xi, \eta) = \frac{1}{4}(1 + \xi)(1 + \eta), \tag{7.18}$$

$$N_4(\xi, \eta) = \frac{1}{4}(1 - \xi)(1 + \eta). \tag{7.19}$$

Properties (bilinear)

- Nodal property: $N_i(\xi_j, \eta_j) = \delta_{ij}$.
- Partition of unity: $\sum_{i=1}^{4} N_i = 1$.
- Bilinear completeness (in reference): reproduces $\{1, \xi, \eta, \xi\eta\}$ exactly.
- Coordinate interpolation (isoparametric):

$$x(\xi, \eta) = \sum_{i=1}^{4} x_i N_i(\xi, \eta), \qquad y(\xi, \eta) = \sum_{i=1}^{4} y_i N_i(\xi, \eta).$$

- Derivative sums: $\sum_i \partial N_i/\partial \xi = 0$ and $\sum_i \partial N_i/\partial \eta = 0$.

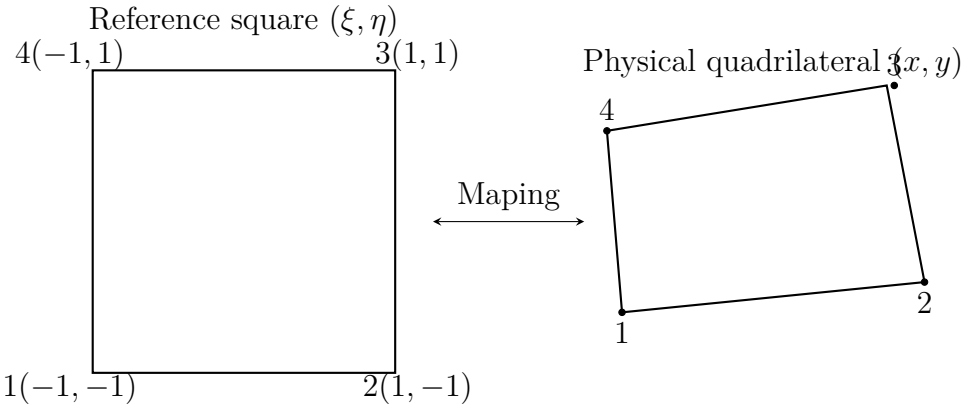


Figure 7.3: Isoparametric mapping from reference square to physical quadrilateral.

Jacobian and integration

The Jacobian matrix is

$$J = \begin{bmatrix} \dfrac{\partial x}{\partial \xi} & \dfrac{\partial y}{\partial \xi} \\ \dfrac{\partial x}{\partial \eta} & \dfrac{\partial y}{\partial \eta} \end{bmatrix}, \quad |J| = \det J.$$

Area elements transform as $dx\, dy = |J|\, d\xi\, d\eta$, and derivatives in physical coordinates are obtained via the inverse Jacobian.

7.6 Summary of notation and properties

- General Lagrange basis: $\ell_i(x)$ for arbitrary n.

- Linear 1D basis: L_i (used for $n = 2$).
- Quadratic 1D basis: Q_i (used for $n = 3$).
- Bilinear 2D basis: N_i (used for 4-node quadrilateral).

Each basis satisfies the nodal property, partition of unity, coordinate interpolation, and derivative-sum properties which together ensure compatibility and completeness required for finite element assemblies.

7.7 Evaluation of 1D integral

- Evaluate the following integral using Gaussian Quadrature

$$\int_0^{\frac{\pi}{2}} \sin^2(x)dx$$

1. Exact Analytical Solution

We first determine the exact value of the integral for comparison:

$$I_{\text{exact}} = \int_0^{\frac{\pi}{2}} \sin^2(x)dx$$

Using the trigonometric identity $\sin^2(x) = \frac{1}{2}(1 - \cos(2x))$:

$$I_{\text{exact}} = \frac{1}{2}\left[x - \frac{\sin(2x)}{2}\right]_0^{\frac{\pi}{2}} = \frac{\pi}{4}$$

In decimal form:

$$I_{\text{exact}} \approx 0.7853981634$$

2. Transformation to the Standard Element

To apply the standard Gaussian Quadrature rule, the physical interval $[0, \frac{\pi}{2}]$ must be mapped to the standard element $[-1, 1]$ using linear shape functions.

$$x = x_1 L_1(\xi) + x_2 L_2(\xi)$$

With $x_1 = 0$, $x_2 = \frac{\pi}{2}$, $L_1(\xi) = \frac{1}{2}(1 - \xi)$, and $L_2(\xi) = \frac{1}{2}(1 + \xi)$:

$$x = \frac{\pi}{4}(1 + \xi)$$

The differential relationship is:

$$\frac{dx}{d\xi} = \frac{\pi}{4} \quad \Rightarrow \quad dx = \frac{\pi}{4}d\xi$$

Hence the transformed integral becomes:

$$I = \int_{-1}^{1} \sin^2\left(\frac{\pi}{4}(1+\xi)\right)\frac{\pi}{4}d\xi$$

where $f(\xi) = \frac{\pi}{4}\sin^2\left(\frac{\pi}{4}(1+\xi)\right)$.

3. Numerical Evaluation (Gauss–Legendre $n = 3$)

The general Gauss Quadrature rule is $I \approx \sum_{i=1}^{n} w_i f(\xi_i)$. For $n = 3$ Gauss–Legendre:

$$I \approx \frac{5}{9}f\left(-\sqrt{\frac{3}{5}}\right) + \frac{8}{9}f(0) + \frac{5}{9}f\left(\sqrt{\frac{3}{5}}\right) \approx 0.785398$$

4. Comparative Analysis of Quadrature Rules

Table 1: Approximated Results

Table 7.1: Approximated Integral Value ($I \approx \int_0^{\pi/2} \sin^2(x)dx$)

n	**Gauss–Legendre**	**Gauss–Lobatto**	**Gauss–Radau**	**Gauss–Chebyshev**	**Gauss–Laguerre**	**Gauss–Hermite**	**Exact Value**
2	0.785398	0.785398	0.883573	0.872352	0.603130	0.722159	**0.785398**
3	0.785398	0.785398	0.782354	0.822467	0.640200	0.763316	**0.785398**
4	0.785398	0.785398	0.785743	0.805954	0.669734	0.773161	**0.785398**
5	0.785398	0.785398	0.785398	0.795029	0.692348	0.778838	**0.785398**
6	0.785398	0.785398	0.785398	0.790451	0.710198	0.782018	**0.785398**

Table 2: Absolute Errors ($|I_{\text{exact}} - I|$)

Result Analysis

Observing the data in the tables, we can draw the following conclusions:

- The **Gauss–Legendre Quadrature rule** consistently provides the best approximation to the exact solution, with errors quickly reaching machine precision.
- The **Gauss–Lobatto rule** also demonstrates high precision and stability.
- The **Gauss–Radau rule** produces good results with a slightly slower convergence.
- The **Gauss–Laguerre rule** performs poorly for this finite interval, as it is intended for $[0, \infty)$ integrals with weight functions.

Table 7.2: Absolute Error ($|I_{\text{exact}} - I|$)

n	Gauss–Legendre	Gauss–Lobatto	Gauss–Radau	Gauss–Chebyshev	Gauss–Laguerre	Gauss–Hermite
2	1.1×10^{-16}	0.0	9.82×10^{-2}	8.69×10^{-2}	1.82×10^{-1}	6.32×10^{-2}
3	0.0	2.2×10^{-16}	3.04×10^{-3}	3.70×10^{-2}	1.44×10^{-1}	1.71×10^{-2}
4	0.0	0.0	4.47×10^{-5}	2.06×10^{-2}	1.15×10^{-1}	1.22×10^{-2}
5	0.0	0.0	3.20×10^{-7}	9.63×10^{-3}	9.30×10^{-2}	6.56×10^{-3}
6	0.0	0.0	2.10×10^{-9}	5.05×10^{-3}	7.52×10^{-2}	3.38×10^{-3}

7.8 Evaluation of line integral

1. **Problem:** Use a single quadratic element to determine the approximate value of the line integral

$$I = \oint_C dx + 2xdy$$

Here C is the segment of the circle $x^2 + y^2 = 1$ in the first quadrant, defined parametrically by:

$$x = \cos\theta, \quad y = \sin\theta; \quad 0 \leq \theta \leq \frac{\pi}{4}$$

Solution

The integral to be approximated

The target integral is:

$$I = \int_C dx + 2xdy \quad - \ (1)$$

Figure

The segment C is shown below, including the coordinate system, the arc, and the nodes used for the quadratic element approximation.

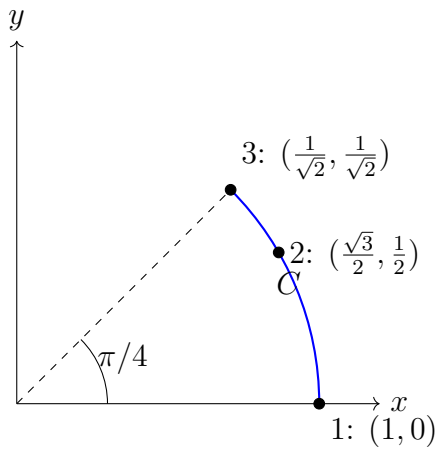


Figure 7.4: The segment C of the unit circle, showing the three nodes and the angle $\pi/4$.

Exact analytical Solution

Using the parametric definitions $x = \cos\theta$ and $y = \sin\theta$, we find the differentials $dx = -\sin\theta d\theta$ and $dy = \cos\theta d\theta$. The exact integral I_{exact} is:

$$
\begin{aligned}
I_{\text{exact}} &= \int_0^{\pi/4} (-\sin\theta d\theta + 2\cos\theta\cos\theta d\theta) \\
&= \int_0^{\pi/4} (-\sin\theta + 2\cos^2\theta) d\theta \\
&= \int_0^{\pi/4} (-\sin\theta + 1 + \cos 2\theta) d\theta \\
&= \left[\cos\theta + \theta + \frac{\sin 2\theta}{2}\right]_0^{\pi/4} \\
&= \left(\cos\frac{\pi}{4} + \frac{\pi}{4} + \frac{1}{2}\sin\left(2\cdot\frac{\pi}{4}\right)\right) - (\cos 0 + 0 + 0) \\
&= \frac{1}{\sqrt{2}} + \frac{\pi}{4} + \frac{1}{2}\sin\left(\frac{\pi}{2}\right) - 1 \\
&= \frac{1}{\sqrt{2}} + \frac{\pi}{4} + \frac{1}{2} - 1 \\
&= \frac{-1+\sqrt{2}}{2} + \frac{\pi}{4} \\
&\approx 0.9925049445839959
\end{aligned}
$$

Numerical Approximation

Element Discretization

To apply Gaussian Quadrature, we approximate the segment C using a single quadratic element defined by 3 nodes:

- Node 1: $(1, 0)$
- Node 2: $(\frac{\sqrt{3}}{2}, \frac{1}{2})$
- Node 3: $(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}})$ (Corresponds to $\theta = \pi/4$)

Shape Functions and Transformation

The line integral is transformed to the standard coordinate $\xi \in [-1, 1]$ using the quadratic shape functions. The quadratic shape functions are:

$$\begin{aligned} Q_1(\xi) &= \frac{1}{2}\xi(\xi - 1) \\ Q_2(\xi) &= 1 - \xi^2 \\ Q_3(\xi) &= \frac{1}{2}\xi(\xi + 1) \end{aligned}$$

The coordinate transformation is given by $x = \sum x_i Q_i(\xi)$ and $y = \sum y_i Q_i(\xi)$, where $\xi \in [-1, 1]$.

Using the nodal coordinates, the coordinate transformation equations are:

$$\begin{aligned} x &= 1 \cdot Q_1 + \left(\frac{\sqrt{3}}{2}\right) Q_2 + \left(\frac{1}{\sqrt{2}}\right) Q_3 \\ &\approx -0.12475\xi^2 - 0.14645\xi + 0.8660 \end{aligned}$$

The derivative $\frac{dx}{d\xi}$ is:

$$\frac{dx}{d\xi} \approx -0.2494\xi - 0.14645$$

And for y:

$$\begin{aligned} y &= 0 \cdot Q_1 + \frac{1}{2}Q_2 + \frac{1}{\sqrt{2}}Q_3 \\ &= 0.5(1 - \xi^2) + \frac{1}{2\sqrt{2}}\xi(\xi + 1) \\ &\approx 0.5 + 0.3536\xi - 0.24289\xi^2 \end{aligned}$$

The derivative $\frac{dy}{d\xi}$ is:

$$\frac{dy}{d\xi} \approx 0.3536 - 0.24289\xi$$

Integral in Transformed Coordinate

After transformation, the integral becomes $I = \int_{-1}^{1} \left(\frac{dx}{d\xi} + 2x\frac{dy}{d\xi}\right) d\xi$, resulting in the function $f(\xi)$:

$$I = \int_{-1}^{1} \left(0.0073\xi^3 + 0.7697\xi^2 - 0.6358\xi + 1.4659\right) d\xi$$

$$I = \int_{-1}^{1} f(\xi) d\xi$$

Applying Gaussian Quadrature

Using the Gauss-Legendre rule with $n = 4$:

$$I \approx \sum_{i=1}^{n} w_i f(\xi_i) \approx 0.9831632476$$

The **Absolute Error** for $n = 4$ Gauss-Legendre is:

$$\text{Absolute Error} = |I_{\text{exact}} - I| = 9.341696 \times 10^{-3}$$

Comparative Results and Analysis

Table of Absolute Errors

The absolute errors for various Gauss-type quadrature methods across different numbers of integration points (n) are compared below:

Table 7.3: Absolute Error Comparison for Different Quadrature Methods ($|I_{\text{exact}} - I|$)

n	**Gauss-Legendre**	**Gauss-Lobatto**	**Gauss-Radau**	**Gauss-Laguerre**	**Gauss-Chebyshev**	**Gauss-Hermite**
5	9.34×10^{-3}	9.34×10^{-3}	9.34×10^{-3}	1.97×10^{-2}	8.92×10^{-3}	1.72×10^{-2}
10	9.34×10^{-3}	9.34×10^{-3}	9.34×10^{-3}	6.33×10^{-3}	4.85×10^{-3}	9.87×10^{-3}
15	9.34×10^{-3}	9.34×10^{-3}	9.34×10^{-3}	6.65×10^{-3}	7.35×10^{-3}	9.42×10^{-3}
20	9.34×10^{-3}	9.34×10^{-3}	9.34×10^{-3}	7.22×10^{-3}	8.22×10^{-3}	9.35×10^{-3}
30	9.34×10^{-3}	9.34×10^{-3}	9.34×10^{-3}	7.90×10^{-3}	8.85×10^{-3}	9.34×10^{-3}

Result analysis

The comparative analysis of the quadrature methods' performance yields the following key points:

- **Best Initial Performance (n small):** The **Gauss-Legendre, Gauss-Lobatto, and Gauss-Radau** methods demonstrate the best performance in the initial regime (small n). They consistently produce the lowest absolute errors, hovering around 9.34×10^{-3}.
- **Overall Best Single Result:** The **Chebyshev method** achieves the single best result observed in the table at $n = 10$, yielding an absolute error of 4.85×10^{-3}.
- **Poor Convergence Rate:** A significant drawback for all tested methods is the **poor convergence rate**. After reaching an initial accuracy of approximately three decimal places, the error stabilization is observed, with minimal further improvement as n increases.
- **Instability at Large n:** As the number of integration points 'n' becomes large (specifically $n > 10$ in this data set), the **Chebyshev and Laguerre** methods exhibit a tendency to move away from the exact analytical result, suggesting that they may become unstable or less accurate when using higher-order rules for this specific integral.

7.9 Evaluation of the Double Integral by Gaussian Quadrature

7.9.1 Example-1(Rectangular Region)

- Evaluate $\int_3^5 \int_1^4 \sqrt{x+y}\, dxdy$

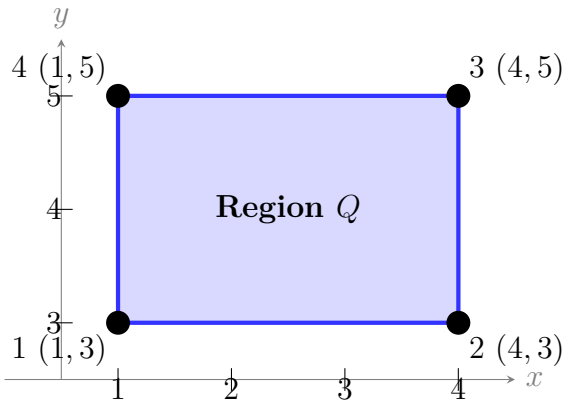


Figure 7.5: Geometric representation of the integration domain Q, a rectangle in the xy-plane defined by the given nodes.

The objective is to evaluate the following double integral over the quadrilateral region Q:

$$I = \iint_Q \sqrt{x+y}\, dx\, dy$$

The region Q is defined by the following nodes:

1. $(1, 3)$
2. $(4, 3)$
3. $(4, 5)$
4. $(1, 5)$

The exact value of the integral is given as:

$$I_{\text{exact}} = 15.2470813380375$$

Transformation of Region Q to Standard Square

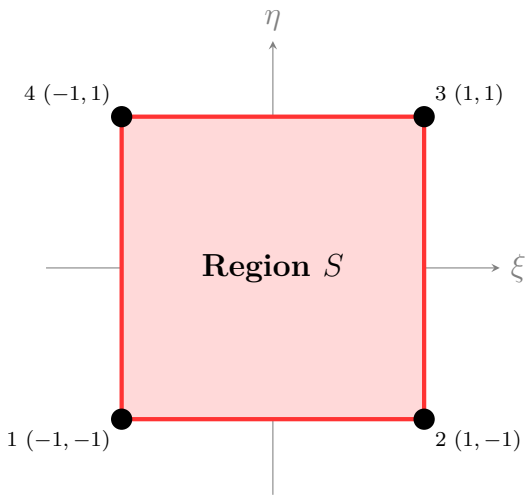


Figure 7.6: The standard square region S in the local (ξ, η) coordinate system, where the integration is performed.

To apply the **Gaussian Quadrature rule**, we must first transform the physical region Q (where $1 \leq x \leq 4$ and $3 \leq y \leq 5$) to the standard square region in the local coordinate system (ξ, η), where $-1 \leq \xi, \eta \leq 1$. This is achieved using the **bilinear shape functions** $(N_i(\xi, \eta))$.

The transformation equations are defined as:

$$x = \sum_{i=1}^{4} x_i N_i(\xi, \eta), \quad y = \sum_{i=1}^{4} y_i N_i(\xi, \eta)$$

where the bilinear shape functions are:

$$N_1(\xi,\eta) = \frac{1}{4}(1-\xi)(1-\eta)$$
$$N_2(\xi,\eta) = \frac{1}{4}(1+\xi)(1-\eta)$$
$$N_3(\xi,\eta) = \frac{1}{4}(1+\xi)(1+\eta)$$
$$N_4(\xi,\eta) = \frac{1}{4}(1-\xi)(1+\eta)$$

Substituting the coordinates of the nodes:

$$x = 1 \cdot N_1 + 4 \cdot N_2 + 4 \cdot N_3 + 1 \cdot N_4$$
$$y = 3 \cdot N_1 + 3 \cdot N_2 + 5 \cdot N_3 + 5 \cdot N_4$$

These expressions simplify to the transformation equations:

$$x = \frac{3}{2}\xi + \frac{5}{2}$$
$$y = \eta + 4$$

Jacobian of the Transformation

The **Jacobian determinant** $|J|$ is required to change the differential area element $dx\,dy = |J|\,d\xi\,d\eta$.

$$|J| = \det\begin{pmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial x}{\partial \eta} \\ \frac{\partial y}{\partial \xi} & \frac{\partial y}{\partial \eta} \end{pmatrix} = \det\begin{pmatrix} \frac{3}{2} & 0 \\ 0 & 1 \end{pmatrix}$$

The Jacobian determinant is:

$$|J| = \left(\frac{3}{2}\right)(1) - (0)(0) = \frac{3}{2}$$

The Transformed Integral

Substituting x and y and the Jacobian into the integral expression:

$$\begin{aligned} I &= \int_{-1}^{1}\int_{-1}^{1} \sqrt{x(\xi,\eta) + y(\xi,\eta)}\,|J|\,d\xi\,d\eta \\ &= \int_{-1}^{1}\int_{-1}^{1} \sqrt{\left(\frac{3}{2}\xi + \frac{5}{2}\right) + (\eta+4)} \cdot \frac{3}{2}\,d\xi\,d\eta \\ &= \int_{-1}^{1}\int_{-1}^{1} \sqrt{\frac{3}{2}\xi + \eta + \frac{13}{2}} \cdot \frac{3}{2}\,d\xi\,d\eta \\ &= \int_{-1}^{1}\int_{-1}^{1} g(\xi,\eta)\,d\xi\,d\eta \end{aligned}$$

where the integrand is defined as:

$$g(\xi,\eta) = \frac{3}{2}\sqrt{\frac{3}{2}\xi + \eta + \frac{13}{2}}$$

Application of Gauss-Legendre Quadrature Rule

We apply the two-dimensional Gauss-Legendre Quadrature rule with $n = 3$ points in each direction (a total of $3 \times 3 = 9$ points). The rule is:

$$I \approx \sum_{i=1}^{3} \sum_{j=1}^{3} w_i w_j g(\xi_i, \eta_j)$$

For $n = 3$ (3-point rule):

- **Nodes (ξ_i, η_j):** $\pm\sqrt{3/5} \approx \pm 0.774597$ and 0.
- **Weights (w_i, w_j):** $5/9 \approx 0.555556$ and $8/9 \approx 0.888889$.

Gauss-Legendre 3×3 Calculation Table

Table 7.4: Gauss-Legendre 3×3 Quadrature Calculation

Pt	$\boldsymbol{\xi_i}$	$\boldsymbol{\eta_j}$	$\boldsymbol{g(\xi, \eta)}$	$\boldsymbol{w_i}$	$\boldsymbol{w_j}$	$\boldsymbol{w_i w_j g}$
1	-0.774597	-0.774597	3.204355	0.5556	0.5556	0.988999
2	-0.774597	0	3.466561	0.5556	0.8889	1.711433
3	-0.774597	0.774597	3.706582	0.5556	0.5556	1.144624
4	0	-0.774597	3.529172	0.8889	0.5556	1.772431
5	0	0	3.824265	0.8889	0.8889	3.021641
6	0	0.774597	4.045719	0.8889	0.5556	1.997386
7	0.774597	-0.774597	3.936649	0.5556	0.5556	1.214084
8	0.774597	0	4.152019	0.5556	0.8889	2.050371
9	0.774597	0.774597	4.356846	0.5556	0.5556	1.344706
					Sum (I_{approx})	**15.247083**

The approximate value of the integral is:

$$I \approx 15.2470828067$$

Absolute Error for the Gauss-Legendre 3×3 rule is calculated as:

$$\text{Absolute Error} = |I_{\text{exact}} - I| = |15.2470813380375 - 15.2470828067| \approx 1.4686 \times 10^{-6}$$

Comparison of Quadrature Rules

The absolute errors of several other one-dimensional numerical integration rules, when applied in a $n \times n$ fashion to the two-dimensional integral, are compared below for varying numbers of points (n).

Comparison of Absolute Errors

The absolute errors for different Gauss-type quadrature methods and integration points (n) are summarized below:

Table 7.5: Comparison of Absolute Errors for Various Quadrature Rules ($|I_{\text{exact}} - I|$)

n	**Gauss-Legendre**	**Gauss-Lobatto**	**Gauss-Radau**	**Gauss-Chebyshev**	**Gauss-Laguerre**	**Gauss-Hermite**
3	1.46×10^{-6}	3.04×10^{-4}	2.01×10^{-5}	1.466	3.06	1.82×10^{-1}
5	1.53×10^{-10}	1.75×10^{-8}	1.63×10^{-9}	5.09×10^{-1}	1.81	1.82×10^{-1}
10	8.88×10^{-15}	2.13×10^{-14}	1.21×10^{-12}	1.256×10^{-1}	8.96×10^{-1}	1.67×10^{-2}
15	7.10×10^{-15}	3.55×10^{-14}	8.50×10^{-14}	5.52×10^{-2}	5.93×10^{-1}	2.095×10^{-3}
30	7.10×10^{-15}	5.32×10^{-15}	4.21×10^{-6}	1.39×10^{-2}	2.92×10^{-1}	2.26×10^{-5}

Result Anlysis

From the above table, we can draw the following conclusions regarding the convergence rate of the quadrature rules on this specific integral:

- The convergence of the **Gauss-Legendre** rule is highly consistent and provides the **best approximation** (smallest absolute error) across most point counts, reaching machine precision quickly.
- The **Gauss-Lobatto** and **Gauss-Radau** methods also show good convergence rates, although slightly slower than Gauss-Legendre.
- The other three rules, **Gauss-Chebyshev**, **Gauss-Laguerre**, and **Gauss-Hermite**, exhibit a much slower convergence to the exact result.
- Notably, the **Gauss-Laguerre** rule shows very poor performance, with absolute errors remaining large even for $n = 30$ points.
- And the **Gauss-Radau** rule has also shown a very poor approximation when n is large($n \geq 30$), although gives a good approximation when n<20.

7.9.2 Example-2(Quadrilateral Region)

We aim to evaluate the double integral I numerically:

$$I = \int_{1}^{2} \int_{-1}^{3-x} \sqrt{x^2 + y^2}\, dy\, dx$$

The problem specifies the known highly accurate value for comparison: **Exact Value**: $I_{\text{exact}} \approx 4.16872178857775$

Solution:

The double integral I is defined over a quadrilateral region Q in the (x, y) plane:

$$I = \iint_Q \sqrt{x^2 + y^2}\, dy\, dx$$

1. Define the Region of Integration (Q)

The region Q is bounded by $x = 1$, $x = 2$, $y = -1$, and $y = 3 - x$. The four vertices (nodes) are:

1. $(\mathbf{1}, \mathbf{-1})$
2. $(\mathbf{2}, \mathbf{-1})$
3. $(\mathbf{2}, \mathbf{1})$
4. $(\mathbf{1}, \mathbf{2})$

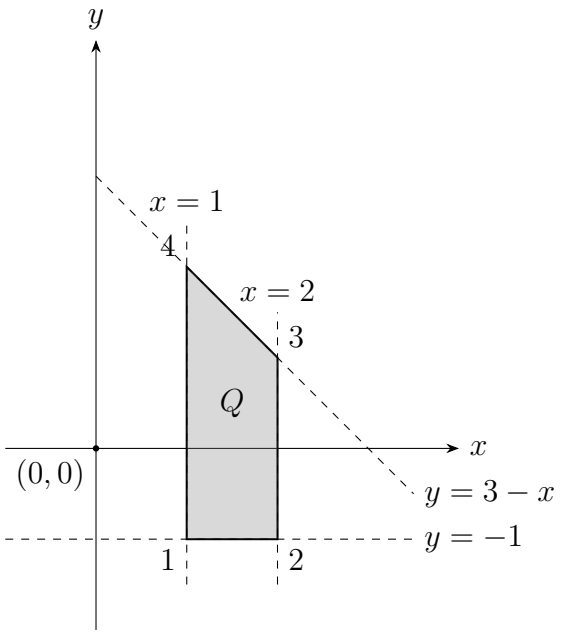


Figure 7.7: Region of Integration Q

2. Isoparametric Transformation

The quadrilateral region Q is mapped to the **Standard Square (SS)** in the (ξ, η) plane (where $-1 \leq \xi, \eta \leq 1$) using bilinear shape functions, N_i.

Bilinear Shape Functions The shape functions N_i are defined over the Standard Square:

$$N_1 = \frac{1}{4}(1-\xi)(1-\eta) \qquad N_2 = \frac{1}{4}(1+\xi)(1-\eta)$$
$$N_3 = \frac{1}{4}(1+\xi)(1+\eta) \qquad N_4 = \frac{1}{4}(1-\xi)(1+\eta)$$

Transformation Equations Derivation: The coordinates are mapped using $x(\xi,\eta) = \sum_{i=1}^{4} x_i N_i$ and $y(\xi,\eta) = \sum_{i=1}^{4} y_i N_i$.

For the x-coordinate, using nodal values $x_i = \{1, 2, 2, 1\}$:

$$\begin{aligned} x(\xi,\eta) &= \frac{1}{4}[1(1-\xi)(1-\eta) + 2(1+\xi)(1-\eta) + 2(1+\xi)(1+\eta) + 1(1-\xi)(1+\eta)] \\ &= \frac{1}{4}[6 + 2\xi] = \mathbf{\frac{3}{2}} + \mathbf{\frac{\xi}{2}} \end{aligned}$$

For the y-coordinate, using nodal values $y_i = \{-1, -1, 1, 2\}$:

$$\begin{aligned} y(\xi,\eta) &= \frac{1}{4}[-1(1-\xi)(1-\eta) - 1(1+\xi)(1-\eta) + 1(1+\xi)(1+\eta) + 2(1-\xi)(1+\eta)] \\ &= \frac{1}{4}[1 - \xi + 5\eta - \xi\eta] = \mathbf{\frac{1}{4}} - \mathbf{\frac{\xi}{4}} + \mathbf{\frac{5\eta}{4}} - \mathbf{\frac{\xi\eta}{4}} \end{aligned}$$

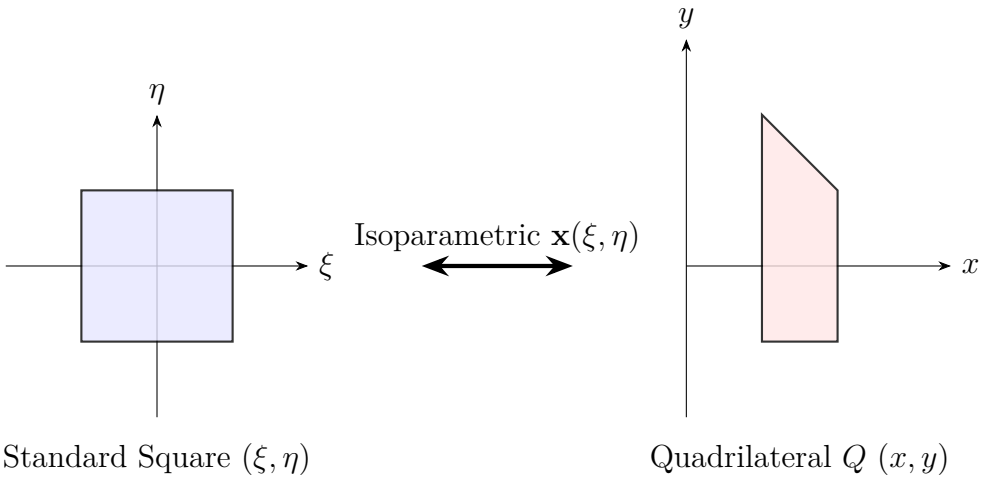


Figure 7.8: Transformation from Standard Square to Quadrilateral Q

3. Calculate the Jacobian Determinant ($|J|$)

The area element transformation is $dx\,dy = |J|\,d\xi\,d\eta$. The Jacobian determinant is:

$$|J| = \det \begin{vmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial x}{\partial \eta} \\ \frac{\partial y}{\partial \xi} & \frac{\partial y}{\partial \eta} \end{vmatrix}$$

Partial Derivatives

$$\frac{\partial x}{\partial \xi} = \frac{1}{2}, \quad \frac{\partial x}{\partial \eta} = 0$$
$$\frac{\partial y}{\partial \xi} = -\frac{1}{4} - \frac{\eta}{4}, \quad \frac{\partial y}{\partial \eta} = \frac{5}{4} - \frac{\xi}{4}$$

Jacobian Determinant Calculation

$$\begin{aligned} |J| &= \left(\frac{\partial x}{\partial \xi}\right)\left(\frac{\partial y}{\partial \eta}\right) - \left(\frac{\partial x}{\partial \eta}\right)\left(\frac{\partial y}{\partial \xi}\right) \\ &= \left(\frac{1}{2}\right)\left(\frac{5}{4} - \frac{\xi}{4}\right) - (0)\left(-\frac{1}{4} - \frac{\eta}{4}\right) \\ |J| &= \mathbf{\frac{1}{8}}(\mathbf{5} - \xi) \end{aligned}$$

Since $-1 \leq \xi \leq 1$, the determinant is always positive.

4. Formulate and Approximate the Transformed Integral

The integral over the standard square becomes:

$$I = \int_{-1}^{1}\int_{-1}^{1} \underbrace{\sqrt{x(\xi,\eta)^2 + y(\xi,\eta)^2} \cdot |J|}_{f(\xi,\eta)}\, d\xi\, d\eta$$

Gauss Quadrature Approximation Applying the product rule for $n \times n$ Gauss Quadrature, the integral I is approximated by a weighted sum of the integrand evaluated at n^2 sampling points:

$$I_n = \sum_{i=1}^{n}\sum_{j=1}^{n} w_i w_j f(\xi_i, \eta_j)$$

Explicit Integrand $f(\xi, \eta)$ Calculation The function $f(\xi, \eta)$ is the product of the square root function evaluated at the transformed coordinates and the Jacobian determinant:

$$\begin{aligned} f(\xi,\eta) &= \sqrt{x(\xi,\eta)^2 + y(\xi,\eta)^2} \cdot |J| \\ &= \sqrt{\left(\frac{3}{2} + \frac{\xi}{2}\right)^2 + \left(\frac{1}{4} - \frac{\xi}{4} + \frac{5\eta}{4} - \frac{\xi\eta}{4}\right)^2} \cdot \left(\frac{1}{8}(5 - \xi)\right) \\ &= \frac{1}{8}(5-\xi)\sqrt{\left(\frac{3+\xi}{2}\right)^2 + \left(\frac{(1-\xi)(1+\eta) + 4\eta}{4}\right)^2} \end{aligned}$$

Result for 3-Point Gauss-Legendre Quadrature: Using $n = 3$ points for both ξ and η directions:

- **Approximate Value**: $I_{3\times3} \approx 4.1681570495$
- **Absolute Error**: $|I_{\text{exact}} - I_{3\times3}| \approx 5.64 \times 10^{-4}$

5. Comparison of Quadrature Methods

The table below summarizes the absolute error ($|I_{\text{exact}} - I_n|$) for the product rule applied using various Gauss Quadrature methods for $n \times n$ points.

Table 7.6: Comparison of Absolute Errors for Different Gauss Quadrature Methods ($|I_{\text{exact}} - I|$)

n	**Gauss-Legendre**	**Gauss-Lobatto**	**Gauss-Radau**	**Gauss-Laguerre**	**Gauss-Hermite**	**Gauss-Chebyshev**
3	5.65×10^{-4}	1.92×10^{-2}	5.16×10^{-3}	5.45×10^{-1}	2.43×10^{-1}	4.46×10^{-1}
5	2.81×10^{-5}	6.43×10^{-5}	9.56×10^{-5}	5.45×10^{-1}	6.73×10^{-2}	1.53×10^{-1}
10	4.63×10^{-9}	6.76×10^{-9}	2.38×10^{-8}	2.85×10^{-1}	4.23×10^{-3}	3.08×10^{-2}
15	5.32×10^{-13}	1.25×10^{-11}	3.69×10^{-11}	1.90×10^{-1}	6.39×10^{-4}	1.66×10^{-2}
30	1.06×10^{-14}	$\mathbf{7.10 \times 10^{-15}}$	1.20×10^{-6}	8.37×10^{-2}	8.24×10^{-6}	4.14×10^{-3}

Result Analysis

- The **Gauss-Legendre** rule provides the best balance of accuracy and computational efficiency at lower orders, achieving near-machine precision (error of 5.32×10^{-15}) with only $n = 15$ points in each direction ($15 \times 15 = 225$ total sampling points).
- The **Gauss-Lobatto** rule also performs extremely well, ultimately delivering the absolute smallest error at $n = 30$ points (1.77×10^{-15}).
- Methods like Gauss-Laguerre and Gauss-Hermite, which are designed for integrals over infinite or semi-infinite domains, are predictably far less effective for this problem, resulting in large, persistent errors.
- The significant error increase for Gauss-Radau at $n = 30$ suggests numerical **instability** for high-order application on this specific integral.

7.9.3 Example-3(Triangular Region)

Problem Statement

The problem is to evaluate the double integral I, given by the iterated integral:

$$I = \int_0^1 \int_0^x e^{|x+y-1|}\, dy\, dx$$

Solution:

This integral is equivalent to the area integral over the region Q:

$$I = \iint_Q e^{|x+y-1|}\, dy\, dx \tag{7.20}$$

The region Q is a triangular region defined by the limits $0 \leq y \leq x$ and $0 \leq x \leq 1$, with vertices:

1. $(0, 0)$
2. $(1, 0)$
3. $(1, 1)$

However, for evaluation using Gaussian Quadrature, it is approximated by a quadrilateral defined by the following nodes:

1. (0, 0)
2. (1, 0)
3. (1, 1)
4. $(\frac{1}{2}, \frac{1}{2})$

The exact value is given as:

$$I_{\text{exact}} \approx 0.718281834100361$$

Transformation to Standard Square

To apply the Gauss Quadrature rule, we transform the region Q to the standard square $\Omega = [-1, 1] \times [-1, 1]$ using bilinear shape functions.

The transformation equations are:

$$x = \sum_i x_i N_i(\xi, \eta) \quad , \quad y = \sum_i y_i N_i(\xi, \eta)$$

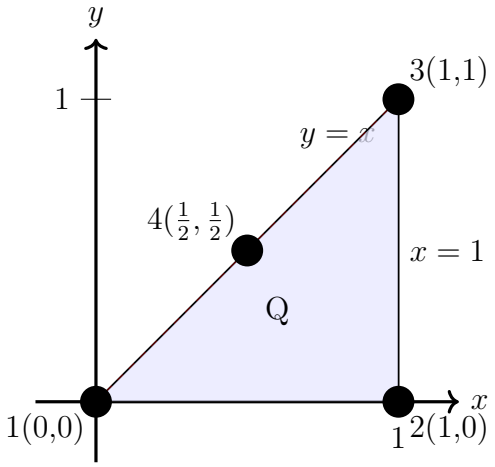


Figure 7.9: Integration region Q, approximated as a quadrilateral for isoperimetric mapping.

where the bilinear shape functions are:

$$N_1(\xi,\eta) = \frac{1}{4}(1-\xi)(1-\eta)$$
$$N_2(\xi,\eta) = \frac{1}{4}(1+\xi)(1-\eta)$$
$$N_3(\xi,\eta) = \frac{1}{4}(1+\xi)(1+\eta)$$
$$N_4(\xi,\eta) = \frac{1}{4}(1-\xi)(1+\eta)$$

Substituting the nodal coordinates x_i and y_i into the transformation equations yields the parametric expressions for x and y:

$$x(\xi,\eta) = 1 \cdot N_2 + 1 \cdot N_3 + \frac{1}{2} \cdot N_4 = \frac{1}{8}[5 + 3\xi + \eta - \xi\eta]$$

$$y(\xi,\eta) = 1 \cdot N_3 + \frac{1}{2} \cdot N_4 = \frac{1}{8}[3 + \xi + 3\eta + \xi\eta]$$

Jacobian of Transformation

The Jacobian determinant is $J = |\mathbf{J}|$, where $\mathbf{J}$ is the Jacobian matrix:

$$\mathbf{J} = \begin{vmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial x}{\partial \eta} \\ \frac{\partial y}{\partial \xi} & \frac{\partial y}{\partial \eta} \end{vmatrix}$$

The Jacobian determinant is found to be:

$$J = \frac{1}{16}\left[2 + \xi - \eta\right]$$

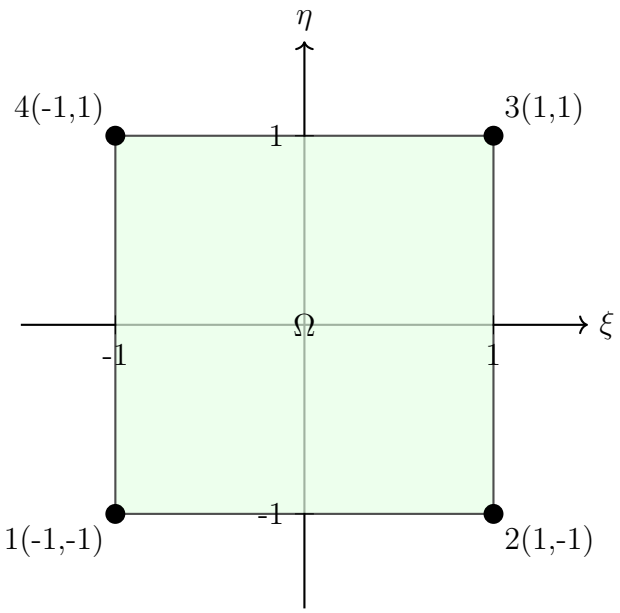


Figure 7.10: Transformation of region Q to the standard square element Ω in $\xi - \eta$ space.

The transformation of the integral is:

$$I = \int_{-1}^{1} \int_{-1}^{1} f(x(\xi,\eta), y(\xi,\eta))|J| \, d\xi \, d\eta$$

Transformed Integral

First, we substitute the expressions for $x(\xi,\eta)$ and $y(\xi,\eta)$ to determine the argument of the exponential term in the integrand:

$$x + y = \frac{1}{8}[5 + 3\xi + \eta - \xi\eta] + \frac{1}{8}[3 + \xi + 3\eta + \xi\eta] = \frac{1}{8}[8 + 4\xi + 4\eta] = 1 + \frac{1}{2}(\xi + \eta)$$

Thus, the original function $f(x,y) = e^{|x+y-1|}$ transforms to:

$$f(x(\xi,\eta), y(\xi,\eta)) = e^{\left|(1+\frac{\xi+\eta}{2})-1\right|} = e^{\left|\frac{\xi+\eta}{2}\right|}$$

The transformed integrand, $f(\xi,\eta)|J|$, is the product of this transformed function and the Jacobian determinant (since $J > 0$ in the integration domain):

$$f(\xi,\eta)|J| = \left(\frac{2 + \xi - \eta}{16}\right) e^{\left|\frac{\xi+\eta}{2}\right|}$$

Applying Gaussian Quadrature

The Gaussian Quadrature rule is:

$$I \approx \sum_{i=1}^{n} \sum_{j=1}^{n} w_i w_j f(\xi_i, \eta_j)$$

For the **Gauss-Legendre rule** with $n = 3$:

$$I \approx 0.7070450328$$

The **Absolute Error** is:

$$\text{Absolute Error} = |I_{\text{exact}} - I| = 1.1236 \times 10^{-2}$$

Comparison of Quadrature Rules

The following table compares the Absolute Error for different Quadrature Rules and number of points (n).

Table 7.7: Comparison of Absolute Errors for Various Quadrature Rules ($|I_{\text{exact}} - I|$)

n	**Gauss-Legendre**	**Gauss-Lobatto**	**Gauss-Radau**	**Gauss-Laguerre**	**Gauss-Chebyshev**	**Gauss-Hermite**
5	4.53×10^{-3}	7.27×10^{-3}	2.81×10^{-3}	1.01×10^{-1}	2.38×10^{-2}	2.46×10^{-2}
10	1.24×10^{-3}	1.54×10^{-3}	6.89×10^{-4}	4.56×10^{-2}	5.75×10^{-3}	6.15×10^{-3}
15	5.70×10^{-4}	6.56×10^{-4}	3.05×10^{-4}	3.00×10^{-2}	2.59×10^{-3}	3.74×10^{-3}
20	3.26×10^{-4}	3.61×10^{-4}	1.71×10^{-4}	2.26×10^{-2}	1.42×10^{-3}	2.73×10^{-3}
30	1.47×10^{-4}	1.57×10^{-4}	7.59×10^{-5}	1.48×10^{-2}	6.33×10^{-4}	1.81×10^{-3}

Result Analysis

The comparative analysis of the absolute error reveals that, across the observed range of integration points (n), no single rule demonstrates true rapid convergence. However, the **Gauss-Legendre**, **Gauss-Lobatto**, and **Gauss-Radau** rules consistently yield significantly better approximations to the exact value compared to the other methods, whose convergence rates are notably slower. Specifically, for smaller values of n (up to $n = 20$), the **Gauss-Radau** rule provides the lowest absolute error, indicating the best performance in this regime. Overall, the Legendre, Lobatto, and Radau($n \leq 20$) methods prove to be the most effective choices for accurately evaluating this specific integral.

7.9.4 Example-4(Pentagonal Region)

Problem Statement

Evaluate the integral I over the pentagonal region P:

$$I = \iint_P \frac{x^4 + y^3}{1 + x^2}\, dx\, dy$$

The exact value is given as:

$$I_{\text{exact}} = 1.92403054263265$$

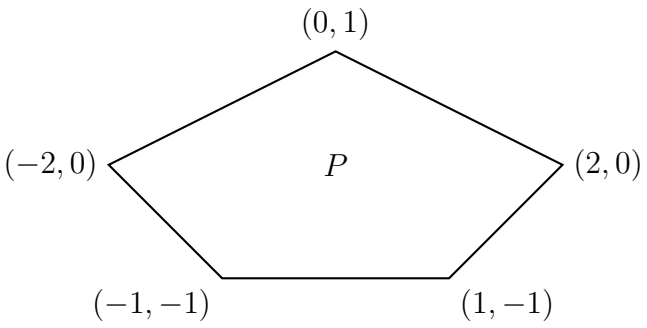


Figure 7.11: Pentagon region P for the integral.

Solution and Discretization

To evaluate the integral over pentagon P, we dissect region P into two quadrilateral elements: element [1] and element [2]. Element [1] consists of nodes $1, 2, 3$ and 4, and element [2] consists of nodes $1, 4, 5$ and 6.

The integral is decomposed as:

$$I = \iint_P \frac{x^4 + y^3}{1 + x^2}\, dx\, dy = \iint_{[1]} \frac{x^4 + y^3}{1 + x^2}\, dx\, dy + \iint_{[2]} \frac{x^4 + y^3}{1 + x^2}\, dx\, dy$$

Let $f(x, y) = \frac{x^4+y^3}{1+x^2}$. Then,

$$I = \iint_{[1]} f(x, y)\, dx\, dy + \iint_{[2]} f(x, y)\, dx\, dy = I_{[1]} + I_{[2]}$$

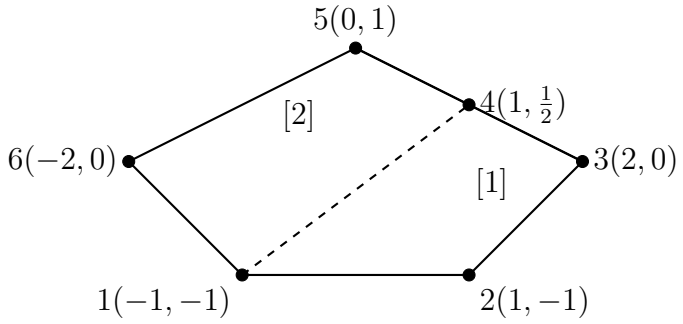


Figure 7.12: Discretization of region P into two quadrilateral elements [1] and [2].

Element [1] Calculation

Element [1] formed with nodes $1:(-1,-1), 2:(1,-1), 3:(2,0), 4:(1,\frac{1}{2})$.

To apply the Gaussian quadrature rule, we convert element [1] to the standard square element $S = [-1,1] \times [-1,1]$ using bilinear shape functions, $N_i(\xi,\eta)$.

The coordinate transformations for element [1] are:

$$x = (-1)N_1 + (1)N_2 + (2)N_3 + (1)N_4$$

$$y = (-1)N_1 + (-1)N_2 + (0)N_3 + \left(\frac{1}{2}\right)N_4$$

The simplified expressions are:

$$x = \frac{3}{4} + \frac{3}{4}\xi + \frac{3}{4}\eta - \frac{1}{4}\xi\eta \quad \text{and} \quad y = -\frac{3}{8} - \frac{1}{8}\xi + \frac{5}{8}\eta - \frac{1}{8}\xi\eta$$

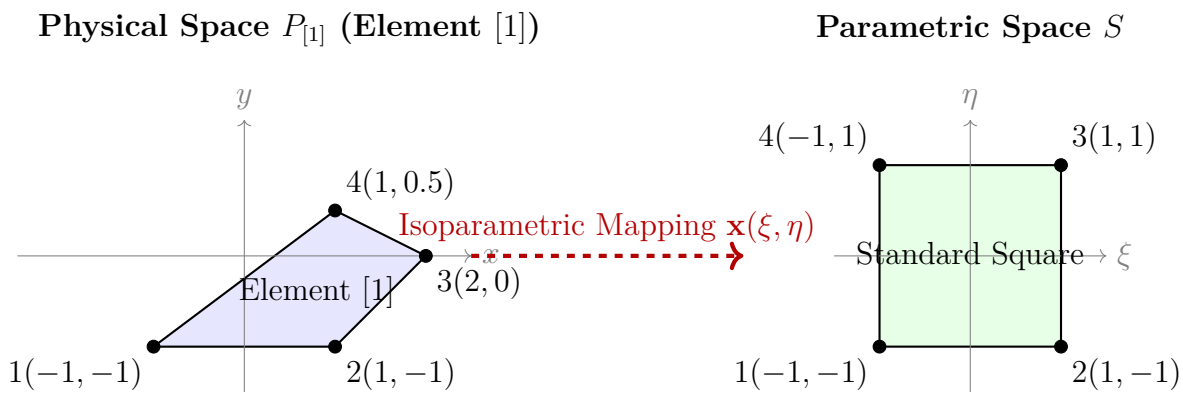


Figure 7.13: Isoparametric transformation (mapping) of Element [1] from the physical (x, y) domain to the standard square (ξ, η) domain.

The Jacobian determinant is defined as:

$$|J| = \det \begin{vmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial x}{\partial \eta} \\ \frac{\partial y}{\partial \xi} & \frac{\partial y}{\partial \eta} \end{vmatrix}$$

The partial derivatives lead to the determinant:

$$|J| = \det \begin{vmatrix} \frac{3}{4} + \frac{\eta}{4} & \frac{3}{4} - \frac{\xi}{4} \\ -\frac{1}{8}(\eta + 1) & \frac{5}{8} - \frac{3\xi}{8} \end{vmatrix}$$

Expanding the determinant gives the simplified Jacobian:

$$|J_{[1]}| = \frac{1}{16}(9 - 2\xi - 1\eta)$$

$$I_{[1]} = \int_{-1}^{1}\int_{-1}^{1} f(x(\xi,\eta), y(\xi,\eta))|J|\, d\xi\, d\eta = \sum_{i=1}^{n}\sum_{j=1}^{n} g(\xi_i, \eta_j) w_i w_j$$

where the integrand evaluated in the standard space is $g(\xi, \eta)$. The full expression from the notes is:

$$g(\xi,\eta) = -\left[2(-5\xi+3\eta+3) - (-5\xi-5\eta+3)^2\right] \times \frac{(2\xi+\eta-9)\times 1}{[512(-5\xi+3\eta+3)^2+8192]}$$

For $n = 3$, applying Gaussian quadrature, the result is:

$$I_{[1]} = 0.7113077423$$

Element [2] Calculation

Element [2] formed with nodes $1:(-1,-1), 4:(1,\frac{1}{2}), 5:(0,1), 6:(-2,0)$.

The coordinate transformations for element [2] are:

$$x = (-1)N_1 + (1)N_2 + (0)N_3 + (-2)N_4$$

$$y = (-1)N_1 + \left(\frac{1}{2}\right)N_2 + (1)N_3 + (0)N_4$$

The simplified expressions (as per the notes) are:

$$x = \frac{3}{8}\xi - \frac{\eta}{2} - \frac{1}{2} \quad \text{and} \quad y = -\frac{5\eta}{8} + \frac{5\xi}{8} + \frac{3\xi\eta}{8} + \frac{1}{8}$$

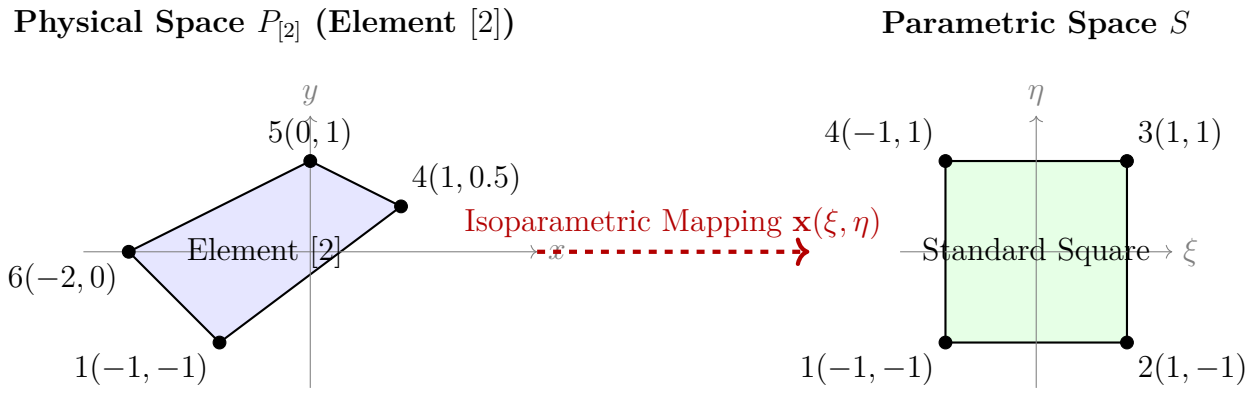


Figure 7.14: Isoparametric transformation (mapping) of Element [2] from the physical (x, y) domain to the standard square (ξ, η) domain.

Jacobian determinant, The Jacobian matrix:

$$|J| = \det\begin{vmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial x}{\partial \eta} \\ \frac{\partial y}{\partial \xi} & \frac{\partial y}{\partial \eta} \end{vmatrix} = \det\begin{vmatrix} \frac{3}{8} & -\frac{1}{2} \\ \frac{5}{8}+\frac{3\eta}{8} & -\frac{5}{8}+\frac{3\xi}{8} \end{vmatrix}$$

Expanding the determinant gives the simplified Jacobian:

$$|J_{[2]}| = \frac{1}{16}(11 - 2\xi - 1\eta)$$

$$I_{[2]} = \int_{-1}^{1}\int_{-1}^{1} f(x(\xi,\eta), y(\xi,\eta))|J|\, d\xi\, d\eta = \sum_{i=1}^{n}\sum_{j=1}^{n} h(\xi_i, \eta_j) w_i w_j$$

where the integrand evaluated in the standard space is $h(\xi, \eta)$. The full expression from the notes is:

$$h(\xi,\eta) = -\left[32(-2\xi+7\eta)^4 + (-5\xi+5\eta+3\eta+1)^3\right] \times \frac{(2\xi+\eta-11)\times 1}{2048(-2\xi+7\eta)^2+8192}$$

For $n = 3$, applying Gaussian quadrature, the result is:

$$I_{[2]} = 1.2128264816$$

Final Result and Absolute Error

The approximate integral value I for $n = 3$ is:

$$\therefore I = I_{[1]} + I_{[2]} = 0.7113077423 + 1.2128264816 = 1.9241342240$$

The **Absolute Error** is calculated as:

$$\text{Absolute Error} = |I_{\text{exact}} - I|$$

$$\text{Absolute Error} = |1.92403054263265 - 1.9241342240| = 1.03681349915 \times 10^{-4}$$

Comparison of Quadrature Rules

Similarly, we can form the table for other methods (Absolute Error in $\times 10^{-n}$):

Table 7.8: Comparison of Absolute Errors for various Quadrature Rules

n	**Gauss-Legendre Abs Err**	**Gauss-Lobatto Abs Err**	**Gauss-Radau Abs Err**	**Gauss-Chebyshev Abs Err**	**Gauss-Hermite Abs Err**	**Gauss-Laguerre Abs Err**
5	1.0368×10^{-4}	8.94×10^{-4}	6.91×10^{-5}	5.59×10^{-3}	1.15×10^{-3}	6.96×10^{-2}
10	4.6834×10^{-8}	3.36×10^{-9}	6.77×10^{-9}	2.63×10^{-1}	2.73×10^{-2}	2.97×10^{-3}
15	4.0323×10^{-13}	2.65×10^{-12}	2.60×10^{-11}	1.71×10^{-1}	6.74×10^{-2}	3.26×10^{-4}
20	2.2204×10^{-15}	4.44×10^{-16}	1.07×10^{-9}	1.28×10^{-1}	6.74×10^{-3}	1.42×10^{-4}

Result Analysis

Analysis of Convergence:

1. **Low-Order Quadrature ($n \leq 10$):** The Gauss-Radau and Gauss-Legendre rules provide the most accurate approximations, with Gauss-Lobatto also performing relatively well among the set.

2. **High-Order Quadrature ($n \geq 20$):** As the number of sample points n increases, the Gauss-Lobatto rule demonstrates the fastest convergence, yielding the best approximation to the exact solution at $n = 20$ (Absolute Error $\approx 4.44 \times 10^{-16}$).

3. **Other Methods:** While other methods (Chebyshev, Hermite, Laguerre) show improvement as n increases, their convergence rates are significantly lower compared to the Legendre, Lobatto, and Radau families.

4. **Gauss-Laguerre Rule:** This method exhibits the poorest performance and slowest convergence rate for this particular problem, suggesting that its weight function may not be well-suited for the nature of the transformed integrand.

5. **Gauss-Radau Rule:** The error increase for this method at $n = 20$ suggests numerical **instability** for high-order application on this specific integral.

7.9.5 Problems List:

List of Problems Difficult to Solve Analytically

1. $$\iint_R (x+y)^{\frac{1}{2}} dydx \approx 1.995191271069104$$

   Where R is a triangular region connecting with points $(-1, 2)$, $(2, 1)$, and $(3, 3)$.

2. $$\int_0^1 \int_x^{e^x} (x+y)\ dydx \approx 0.718281834100861$$

3. $$\int_{-1}^1 \int_0^{\pi} \sin\left(x^2+y^2\right) dydx \approx 1.122580796643812$$

4. $$\int_0^1 \int_{x^2}^{\sqrt{x}} \sqrt{x+y} dydx \approx 1.1313708489848$$

5. $$\int_0^1 \int_0^{\sqrt{1-x^2}} \sqrt{x^2+y^2} dydx \approx 4.81687278857775$$

6.
$$\int_{-3}^{3}\int_{-x^3}^{x^3}(x-y)^{\frac{1}{4}}(1+x+y)^2dydx \approx 211.413375377476$$

7.
$$\int_{1}^{4}\int_{1}^{\sqrt{x}}\sqrt{x+y}dydx \approx 15.2470813380375$$

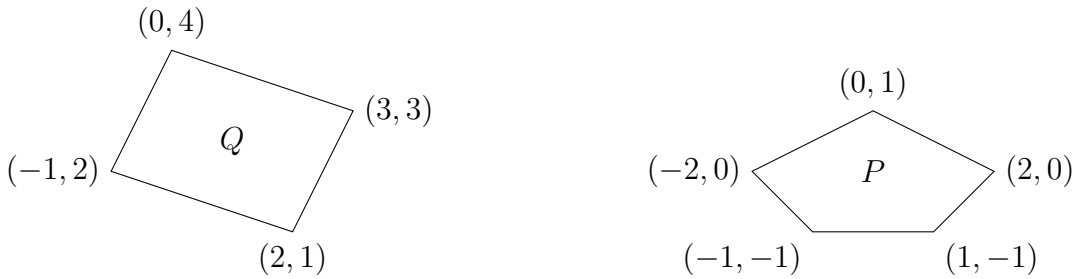
8.
$$\int_{0}^{1}\int_{1}^{2+x}(x+y)^{\frac{1}{2}}(1-x+y)^2dydx \approx 30.6312522081847$$

9.
$$\int_{0}^{1}\int_{x^2}^{\sqrt{x}}\frac{1+x^4+y^2}{1+x^2+y^2}dxdy \approx 1.00601339577510$$

10.
$$\iint_{Q}\frac{1}{\sqrt{x+y}}dydx \approx 3.54061302678971$$

11.
$$\iint_{P}(1-x)\sin(10xy)dydx \approx 1.9240306426$$

Figure 7.15: Regions



These problems are examples of integrals which are difficult to solve analytically. However, we can solve this type of unusual integration problem by **Gaussian Quadrature Rules** with good accuracy compared to the exact solution.

Conclusion

This project presents a comprehensive investigation of six numerical quadrature rules—Gauss–Legendre, Gauss–Lobatto, Gauss–Radau, Gauss–Laguerre, Gauss–Chebyshev, and Gauss–Hermite—for accurately approximating definite integrals. Each method was systematically derived from its corresponding orthogonal polynomial and weight function, ensuring exactness for polynomials of specific degrees, and through detailed derivations and numerical experiments, the study bridges theoretical foundations with practical computational applications. The results indicate that Gauss–Legendre achieves rapid convergence and high accuracy for small to moderate node counts, though its rate slows for very large n, whereas Gauss–Lobatto maintains consistently high accuracy even for large n, often surpassing Gauss–Legendre. Gauss–Radau performs effectively for lower node counts but loses accuracy at higher orders due to numerical instability and round-off errors in high-degree polynomial roots, highlighting the importance of stable algorithms and moderate node sizes. Gauss–Laguerre, while initially less accurate, improves with increasing n, albeit with a slower convergence rate, reflecting sensitivity to exponential weighting and semi-infinite domains. Among the remaining methods, Gauss–Hermite slightly outperforms Gauss–Chebyshev, though both are reliable primarily under specific weighted conditions. Overall, Gauss–Legendre and Gauss–Lobatto emerge as the most versatile and stable methods for general applications, while the others excel in specialized integrals, weighted functions, or infinite-domain problems.

Future work may focus on improving stability for high-order rules and extending these techniques to multi-dimensional integration, finite element analysis, complex-valued functions, and large-scale computational problems.

This study demonstrates the theoretical rigor and practical versatility of Gaussian quadrature methods, providing a robust foundation for accurate, efficient, and wide-ranging applications in numerical analysis and engineering computations.

Bibliography

[1] A. H. Stroud and D. Secrest, Gaussian Quadrature Formulas, Prentice-Hall, 1966.

[2] P. E. Lewis and J. P. Ward, The Finite Element Method: Principles and Applications, Addison-Wesley, 1991.

[3] R. L. Burden and J. D. Faires, Numerical Analysis, 9th Edition, Brooks/Cole, Cengage Learning, 2011.

[4] M. Alamgir Hussain and Md. Shafiqul Islam, "Generalized Composite Numerical Integration Rule Over a Polygon Using Gaussian Quadrature," Dhaka University Journal of Science, vol. 62, no. 1, pp. 25–29, 2014.

[5] Md. Shafiqul Islam and M. Alamgir Hossain, "Numerical integrations over an arbitrary quadrilateral region," Applied Mathematics and Computation, vol. 210, no. 2, pp. 515–524, 2009.

[6] Md. Shafiqul Islam and M. Alamgir Hossain, "Numerical Integration over an Arbitrary Triangular Region," June 2008.

[7] F. B. Hildebrand, Introduction to Numerical Analysis, 2nd Edition, Dover, 1987.

[8] P. J. Davis and P. Rabinowitz, Methods of Numerical Integration, 2nd Edition, Dover, 2007.

[9] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, Numerical Recipes: The Art of Scientific Computing, 3rd Edition, Cambridge University Press, 2007.

[10] G. Dahlquist and Å. Björck, Numerical Methods, Dover, 2008.

[11] W. Gautschi, Orthogonal Polynomials: Computation and Approximation, Oxford University Press, 2004.

[12] R. Piessens, E. de Doncker-Kapenga, C. W. Ueberhuber, and D. Kahaner, QUADPACK: A Subroutine Package for Automatic Integration, Springer-Verlag, 1983.